



SIMATS
ENGINEERING



SIMATS
Savitribi Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DSA02 – COMPUTER VISION with OPENCV

(Academic Year 2026-2027)

Name : V G ABBIRAMY

Reg No : 192524097

Department : B.Tech Artificial Intelligence & data Science



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

REGISTER NUMBER : 192524097

NAME OF THE LAB : DSA02- COMPUTER VISION with OPEN CV

DEPARTMENT : B.Tech Artificial Intelligence & Data Science

Certified that this is a Bonafide Record of Practical Record Work done

by Ms. V G ABBIRAMY of B.Tech Artificial Intelligence & Data Science Department, Sixth

Semester in the **DSA02- Computer Vision with Open CV** Laboratory During the year 2026 - 27.

Signature of Lab-in-Charge

Signature of Head of the Department

Submitted for the Saveetha University Practical Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

INDEX

S.No	Date	Name of the Experiment	Page No.	Sign
1		Image Reading and Grayscale Conversion		
2		Image Blurring Using Gaussian Blur		
3		Edge Detection Using Canny Function		
4		Image Dilation		
5		Image Erosion		
6		Video Processing – Slow/Fast Motion		
7		Webcam Video Capture		
8		Image Scaling		
9		Image Rotation		
10		Image Translation		
11		Affine Transformation		
12		Perspective Transformation – Image		
13		Perspective Transformation – Video		
14		Homography Transformation		
15		Direct Linear Transformation		
16		Canny Edge Detection		
17		Sobel Edge Detection – X Axis		
18		Sobel Edge Detection – Y Axis		
19		Sobel Edge Detection – XY Axis		
20		Laplacian Sharpening – Negative Centre		
21		Laplacian Sharpening – Diagonal Neighbours		
22		Laplacian Sharpening – Positive Centre		
23		Unsharp Masking		
24		High-Boost Masking		
25		Gradient Masking		
26		Image Watermarking		
27		Cropping, Copying and Pasting		
28		Boundary Detection Using Convolution		
29		Morphological Erosion		
30		Morphological Dilation		
31		Morphological Opening		
32		Morphological Closing		
33		Morphological Gradient		
34		Top Hat Transformation		
35		Black Hat Transformation		

36		Watch/Object Recognition		
37		Reverse Video		
38		Face Detection		
39		Vehicle Detection		
40		Rectangular Object Extraction		

Ex. No: 1	Image Reading and Grayscale Conversion
Date:	

Aim:

To read an image using Python and convert the image into grayscale using OpenCV.

Algorithm:

1. Import OpenCV.
2. Read the input image.
3. Convert the image from BGR to grayscale.
4. Display the original and grayscale images.
5. ☐ Close all windows.

Program:

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

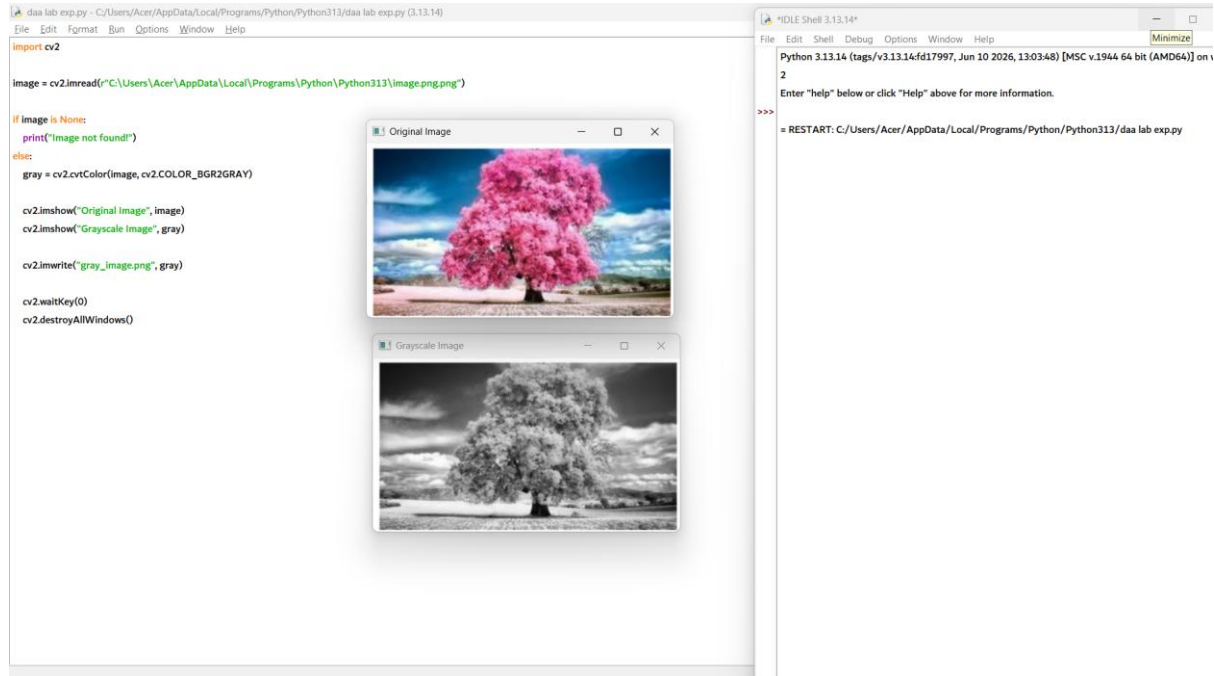
```
cv2.imshow("Original Image", img)
```

```
cv2.imshow("Grayscale Image", gray)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the image was successfully read and converted into grayscale using OpenCV.

Ex. No: 2	Image Blurring Using Gaussian Blur
Date:	

Aim:

To read an image and convert it into a blurred image using Gaussian Blur.

Algorithm

1. Import OpenCV.
2. Read the input image.
3. Apply Gaussian Blur using a suitable kernel.
4. Display the original and blurred images.
5. Close all windows.

Program:

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
blur = cv2.GaussianBlur(img, (5, 5), 0)
```

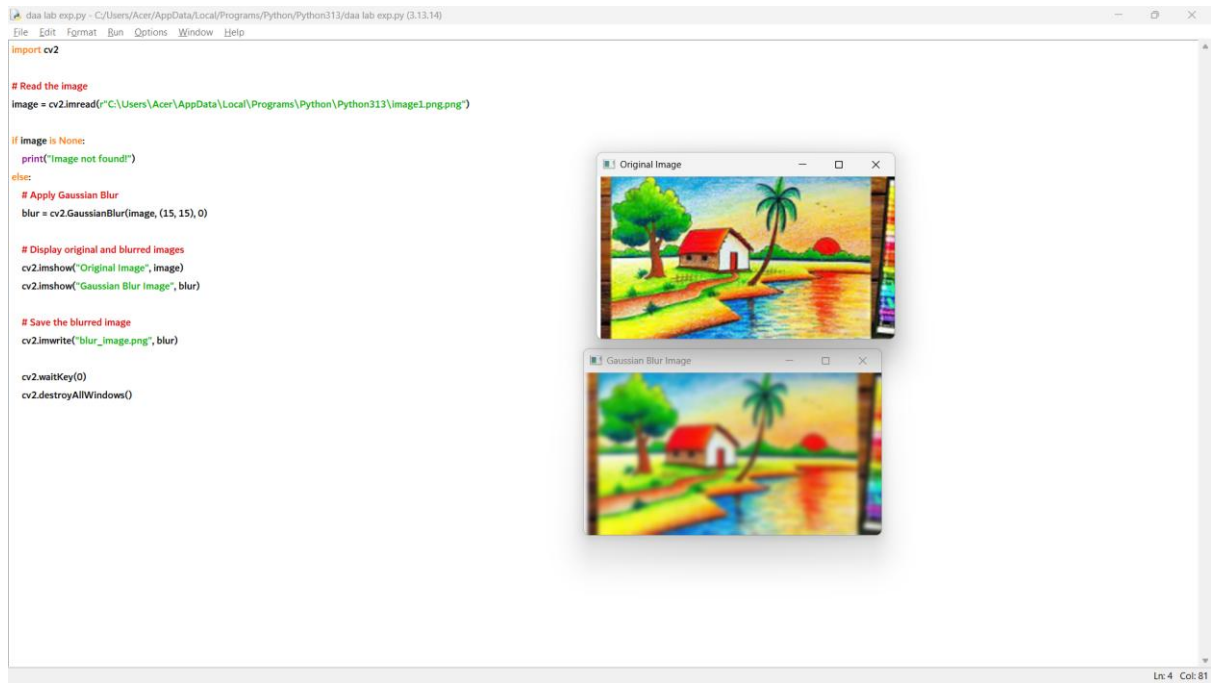
```
cv2.imshow("Original Image", img)
```

```
cv2.imshow("Gaussian Blur", blur)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output:



Result:

Thus, Gaussian Blur was successfully applied to the image.

Ex. No: 3	Edge Detection Using Canny Function
Date:	

Aim:

To read an image and show its outline using the Canny edge detection function.

Algorithm:

1. Read the input image.
2. Convert it into grayscale.
3. Apply Canny edge detection.
4. Display the detected edges.

Program:

```
import cv2

img = cv2.imread("image.jpg")

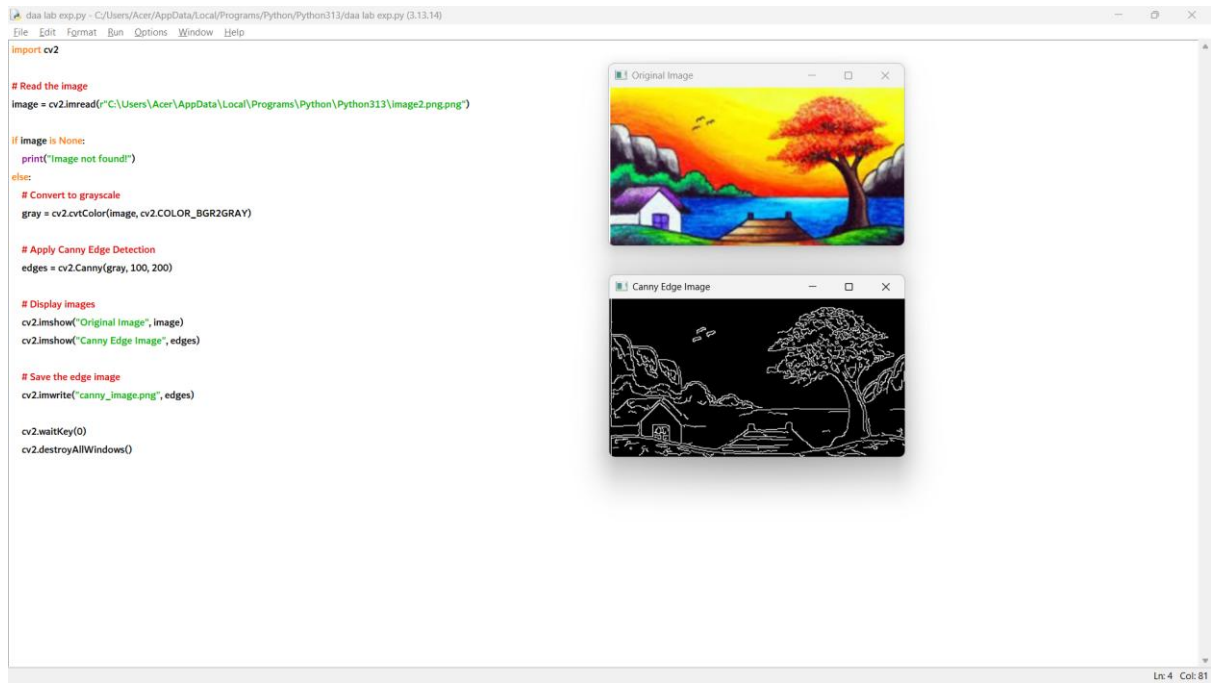
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

cv2.imshow("Original", img)
cv2.imshow("Canny Edges", edges)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the image outline was successfully detected using the Canny function.

Ex. No: 4	Image Dilation Using OpenCV
Date:	

Aim:

To read an image and dilate the image using the dilation operation.

Algorithm:

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring kernel.
4. Apply dilation.
5. Display the dilated image.

Program:

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

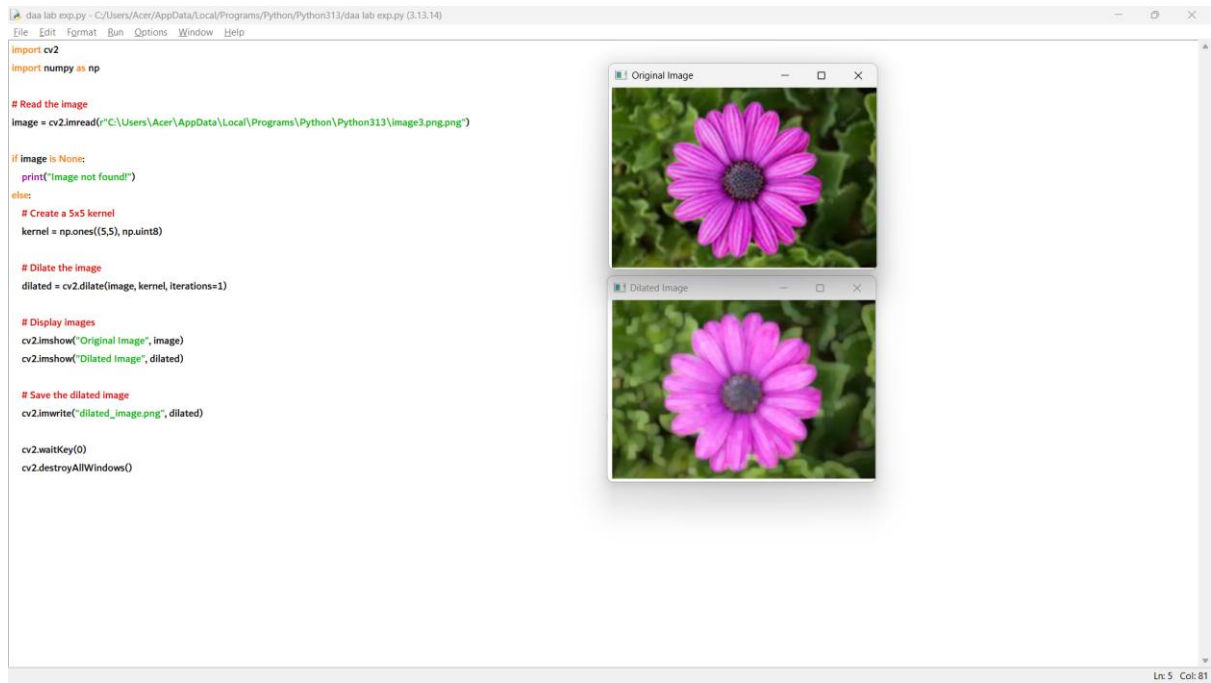
kernel = np.ones((5, 5), np.uint8)

dilated = cv2.dilate(gray, kernel, iterations=1)

cv2.imshow("Original", img)
cv2.imshow("Dilated Image", dilated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the image was successfully dilated using OpenCV.

Ex. No: 5	Image Erosion Using OpenCV
Date:	

Aim:

To read an image and erode the image using the erosion operation.

Algorithm:

1. Read the input image.
2. Convert it into grayscale.
3. Create a structuring kernel.
4. Apply erosion.
5. Display the eroded image.

Program:

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

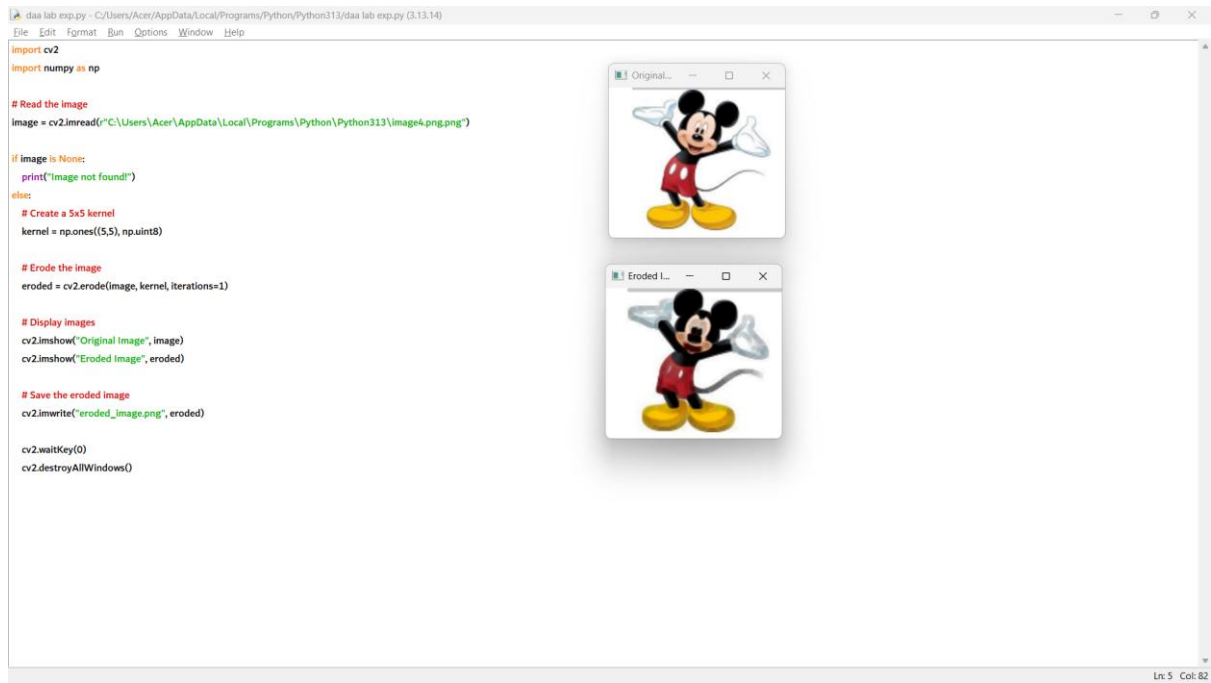
kernel = np.ones((5, 5), np.uint8)

eroded = cv2.erode(gray, kernel, iterations=1)

cv2.imshow("Original", img)
cv2.imshow("Eroded Image", eroded)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the image was successfully eroded using OpenCV.

Ex. No: 6	Video Processing in Slow Motion and Fast Motion
Date:	

Aim:

To read a captured video in Python and display the video in slow motion and fast motion.

Algorithm:

1. Open the video file.
2. Read the video frame by frame.
3. Display each frame.
4. Change the delay between frames to control speed.
5. Release the video after completion.

Program:

```
import cv2

cap = cv2.VideoCapture("video.mp4")

while True:
    ret, frame = cap.read()

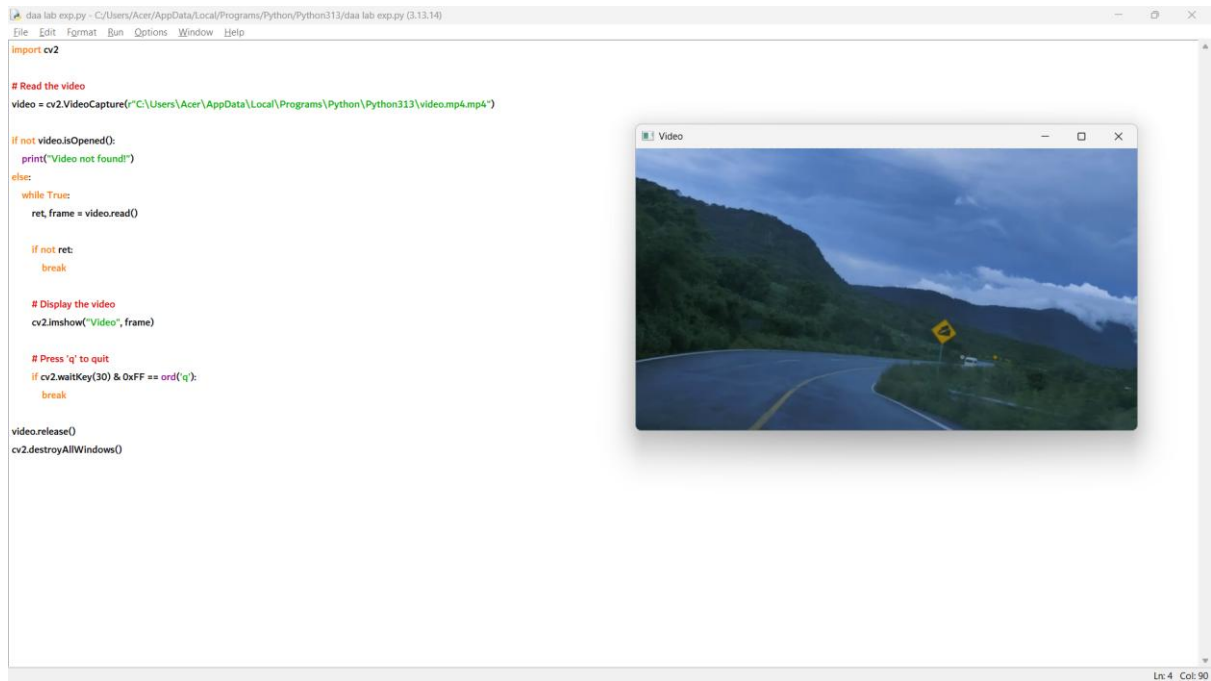
    if not ret:
        break

    cv2.imshow("Video", frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the captured video was successfully processed using OpenCV.

Ex. No: 7	Webcam Video Capture and Speed Control
Date:	

Aim

To capture video from a web camera and display the video in slow motion and fast motion.

Algorithm

1. Access the webcam.
2. Capture video frames.
3. Display the captured frames.
4. Modify the delay to control playback speed.
5. Stop the camera when q is pressed.

Program:

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

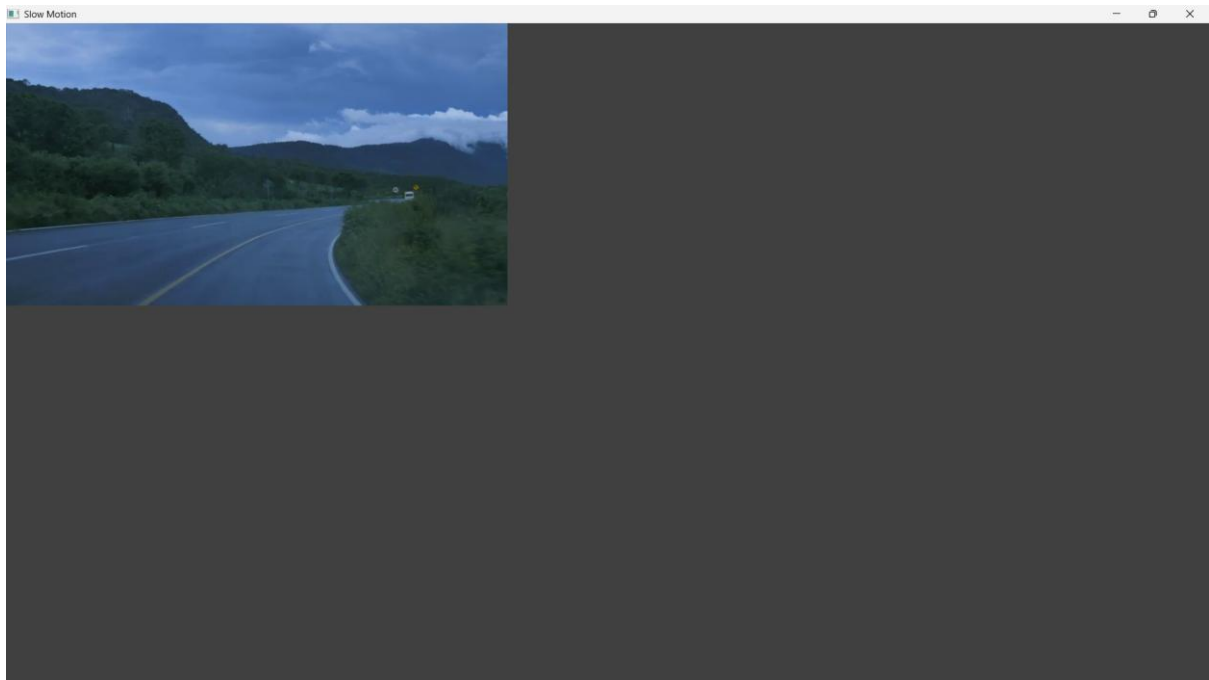
    cv2.imshow("Web Camera", frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

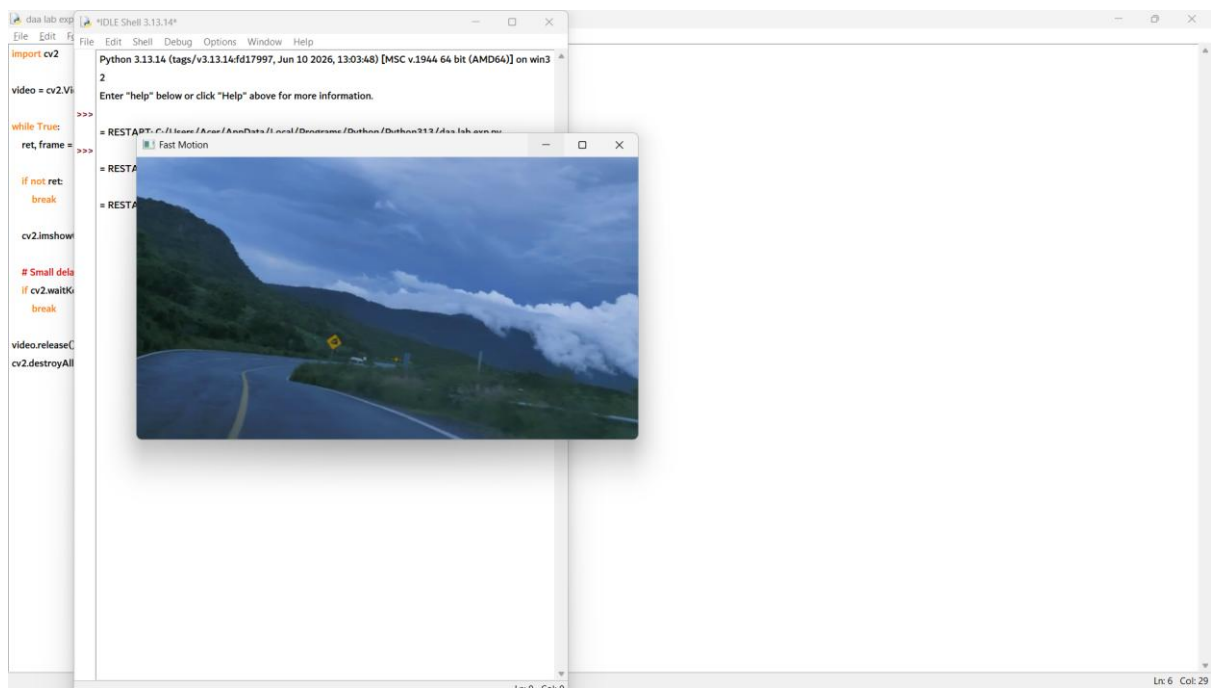
cap.release()
cv2.destroyAllWindows()
```

Output:

Slowmotion:



Fast Motion:



RESULT

Thus, video was successfully captured from the webcam using OpenCV.

Ex. No: 8	Image Scaling – Enlargement and Reduction
Date:	

Aim

To scale an image into bigger and smaller sizes.

Algorithm

1. Read the image.
2. Increase the image dimensions.
3. Decrease the image dimensions.
4. Display the original, enlarged and reduced images.

Program

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
bigger = cv2.resize(img, None, fx=2, fy=2)
```

```
smaller = cv2.resize(img, None, fx=0.5, fy=0.5)
```

```
cv2.imshow("Original", img)
```

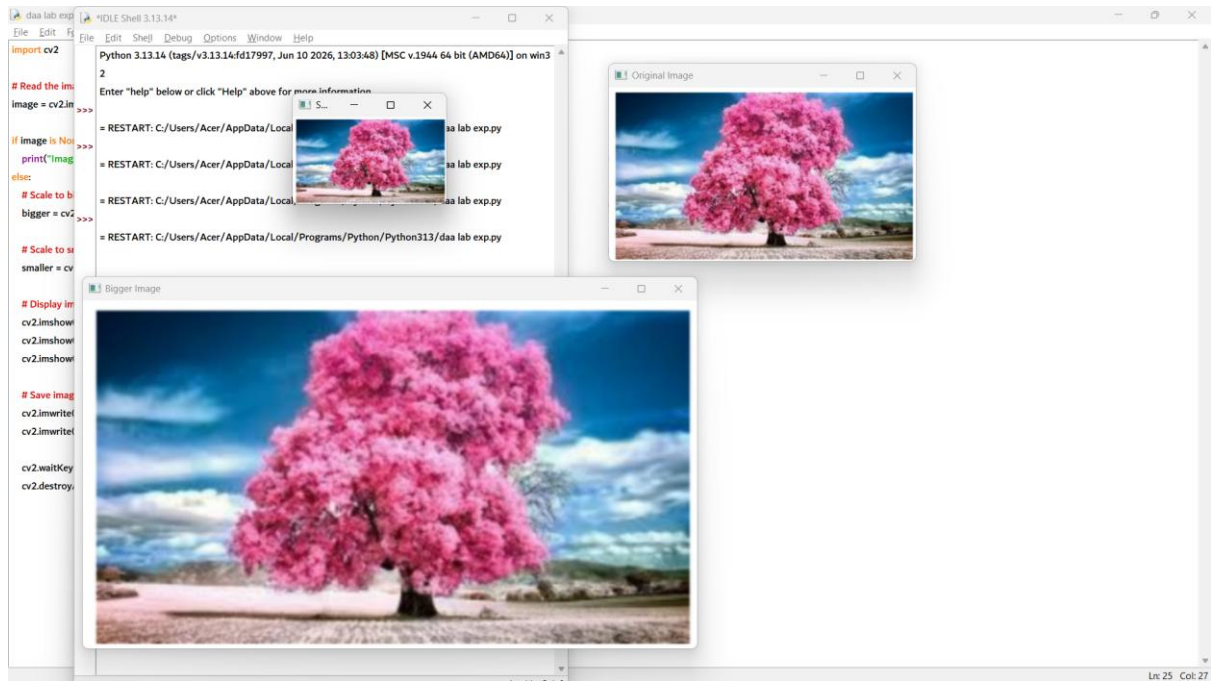
```
cv2.imshow("Bigger", bigger)
```

```
cv2.imshow("Smaller", smaller)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, the image was successfully scaled into bigger and smaller sizes.

Ex. No: 9	Image Rotation – Clockwise and Counter-Clockwise
Date:	

Aim

To rotate an image in clockwise and counter-clockwise directions.

Algorithm

1. Read the input image.
2. Rotate the image clockwise.
3. Rotate the image counter-clockwise.
4. Display both results.

Program

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
clockwise = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

```
counter = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)
```

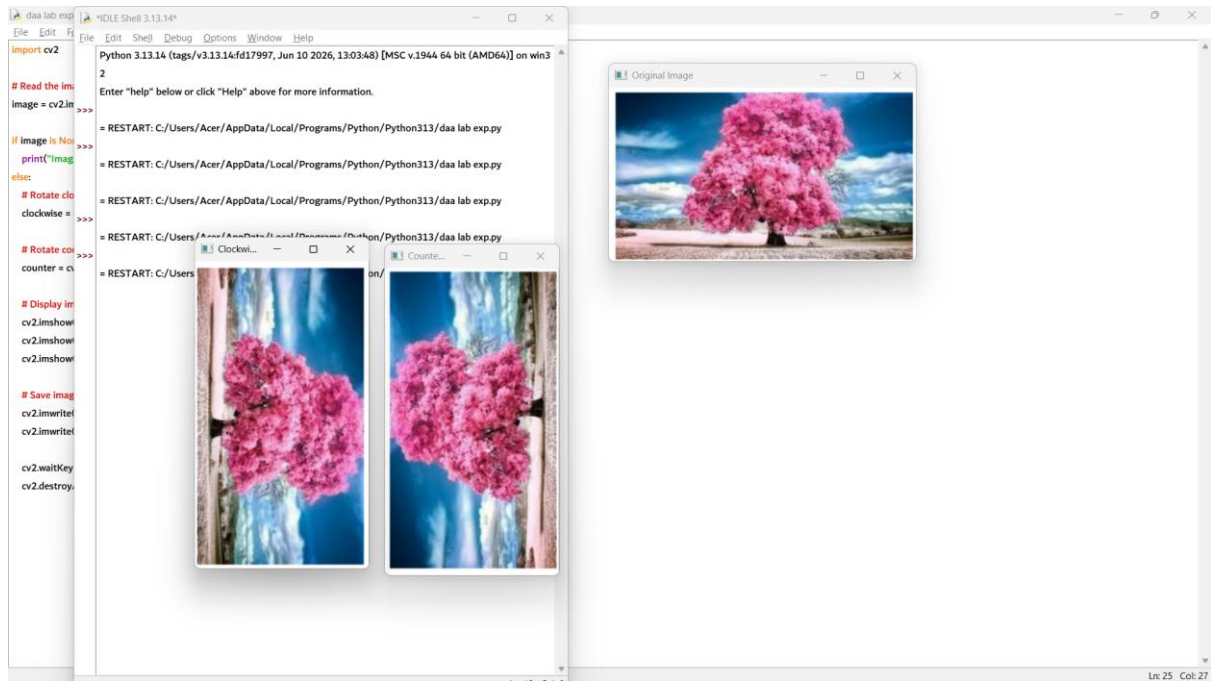
```
cv2.imshow("Clockwise", clockwise)
```

```
cv2.imshow("Counter Clockwise", counter)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, the image was successfully rotated clockwise and counter-clockwise.

Ex. No: 10	Image Translation
Date:	

Aim

To move an image from one position to another using translation transformation.

Algorithm

1. Read the image.
2. Define the translation matrix.
3. Specify horizontal and vertical displacement.
4. Apply translation using `warpAffine()`.
5. Display the translated image.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

matrix = np.float32([[1, 0, 100],
                    [0, 1, 50]])

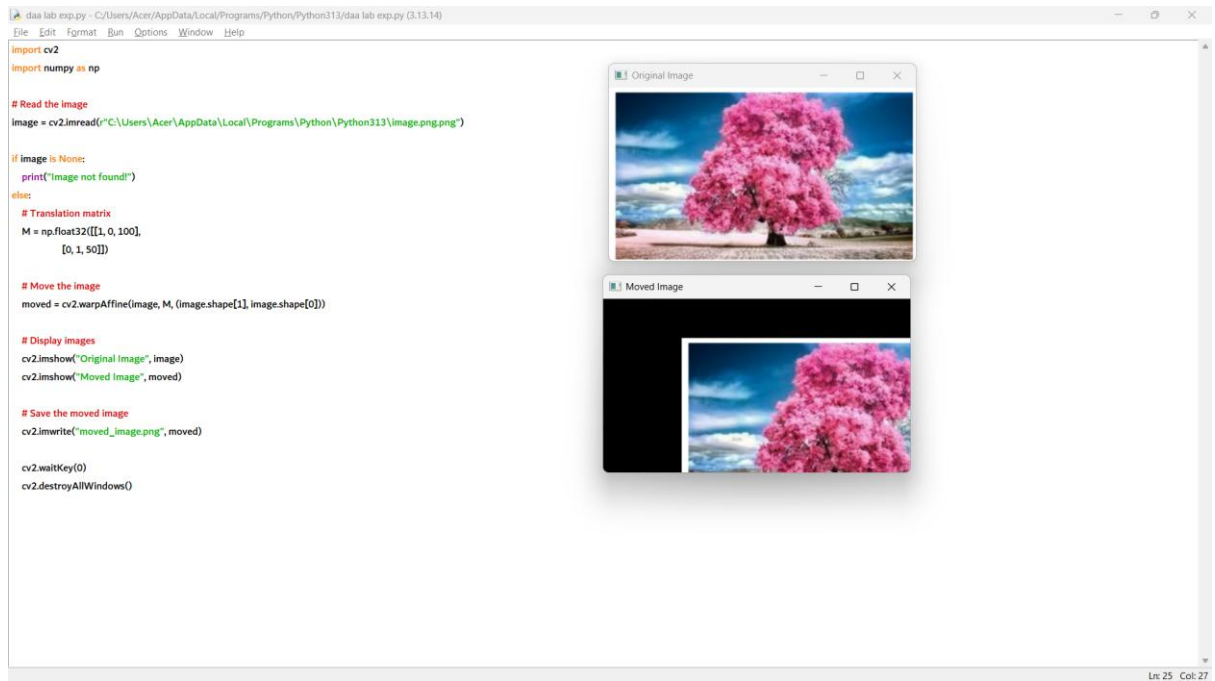
h, w = img.shape[:2]

translated = cv2.warpAffine(img, matrix, (w, h))

cv2.imshow("Original", img)
cv2.imshow("Translated", translated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image translation was successfully performed.

Ex. No: 11	Affine Transformation
Date:	

Aim

To perform affine transformation on an image using OpenCV.

Algorithm

1. Read the image.
2. Select three source points.
3. Select three destination points.
4. Calculate the affine transformation matrix.
5. Apply the transformation.
6. Display the result.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")

src = np.float32([[50,50], [200,50], [50,200]])
dst = np.float32([[10,100], [200,50], [100,250]])

matrix = cv2.getAffineTransform(src, dst)

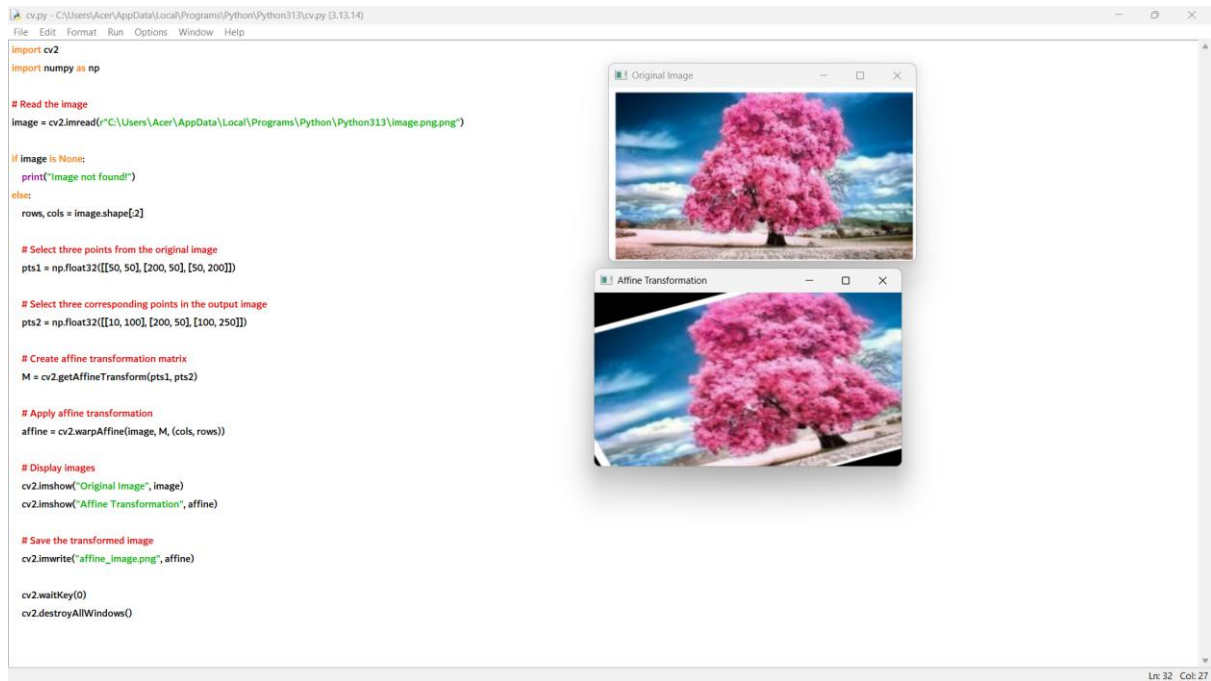
h, w = img.shape[:2]

result = cv2.warpAffine(img, matrix, (w, h))

cv2.imshow("Affine Transformation", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, affine transformation was successfully performed.

Ex. No: 12	Perspective Transformation of Image
Date:	

Aim

To perform perspective transformation on an image using OpenCV.

Algorithm

1. Read the image.
2. Select four source points.
3. Select four destination points.
4. Calculate the perspective matrix.
5. Apply warpPerspective().
6. Display the result.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")
src = np.float32([[50,50], [300,50],
                  [50,300], [300,300]])

dst = np.float32([[0,0], [300,0],
                  [0,300], [300,300]])

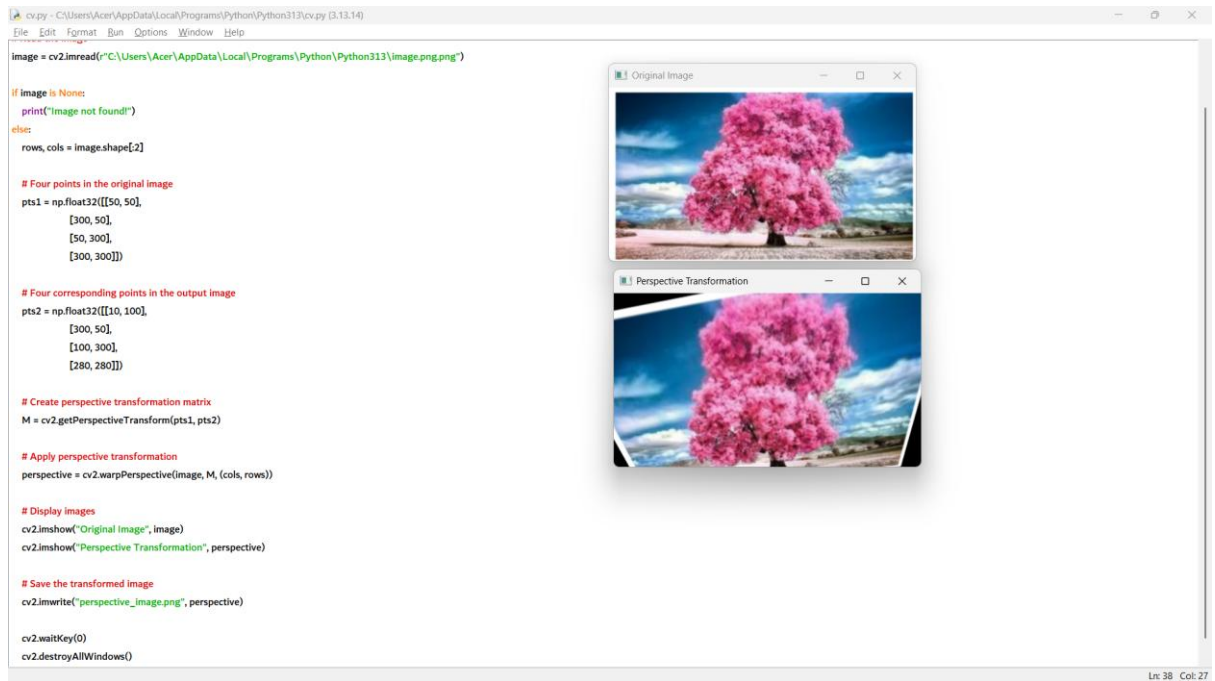
matrix = cv2.getPerspectiveTransform(src, dst)

result = cv2.warpPerspective(img, matrix, (300,300))

cv2.imshow("Perspective", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, perspective transformation was successfully performed.

Ex. No: 13	Perspective Transformation of Video
Date:	

Aim

To perform perspective transformation on a video using OpenCV.

Algorithm

1. Open the input video.
2. Read each frame.
3. Select source and destination points.
4. Calculate the perspective matrix.
5. Apply the transformation to every frame.
6. Display the transformed video.

Program

```
import cv2
import numpy as np

cap = cv2.VideoCapture("video.mp4")

src = np.float32([[50,50], [400,50],
                  [50,300], [400,300]])

dst = np.float32([[0,0], [400,0],
                  [0,300], [400,300]])

matrix = cv2.getPerspectiveTransform(src, dst)

while True:
    ret, frame = cap.read()

    if not ret:
```

```
break
```

```
result = cv2.warpPerspective(frame, matrix, (400,300))
```

```
cv2.imshow("Perspective Video", result)
```

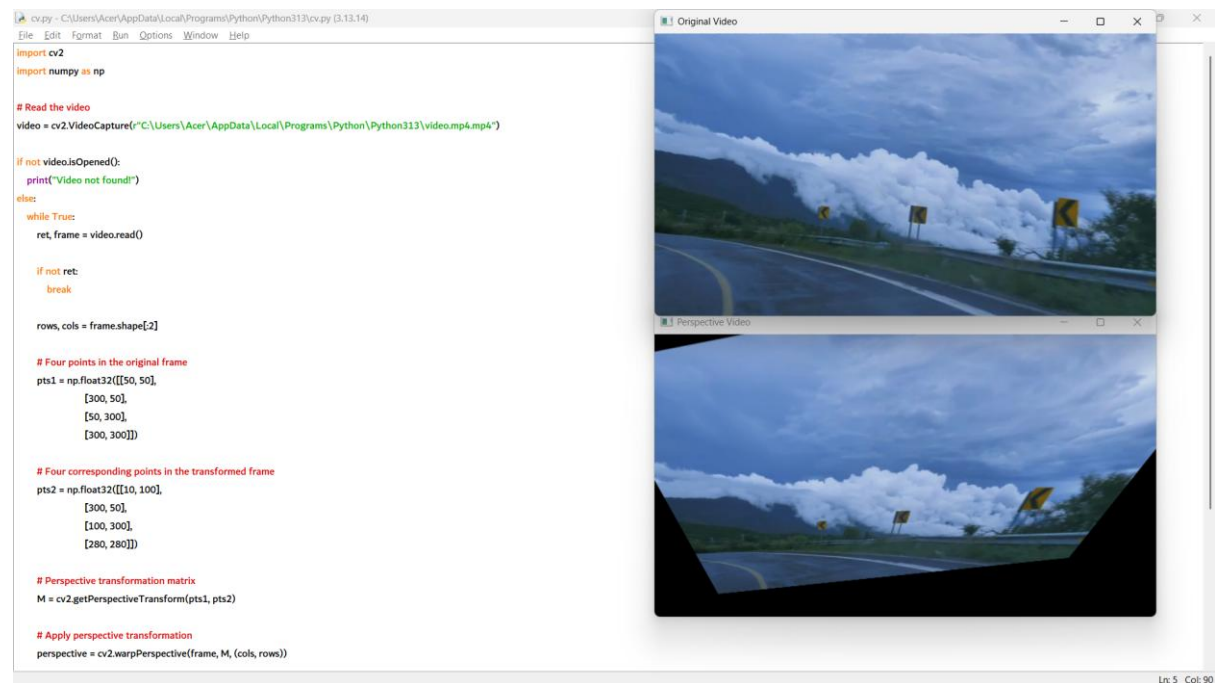
```
if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, perspective transformation was successfully performed on the video.

Ex. No: 14	Image Transformation Using Homography Matrix
Date:	

Aim

To perform image transformation using a Homography matrix.

Algorithm

1. Read the image.
2. Select four corresponding points.
3. Calculate the Homography matrix.
4. Apply the matrix using warpPerspective().
5. Display the transformed image.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")

src = np.float32([[50,50], [300,50],
                  [50,300], [300,300]])

dst = np.float32([[0,0], [300,0],
                  [0,300], [300,300]])

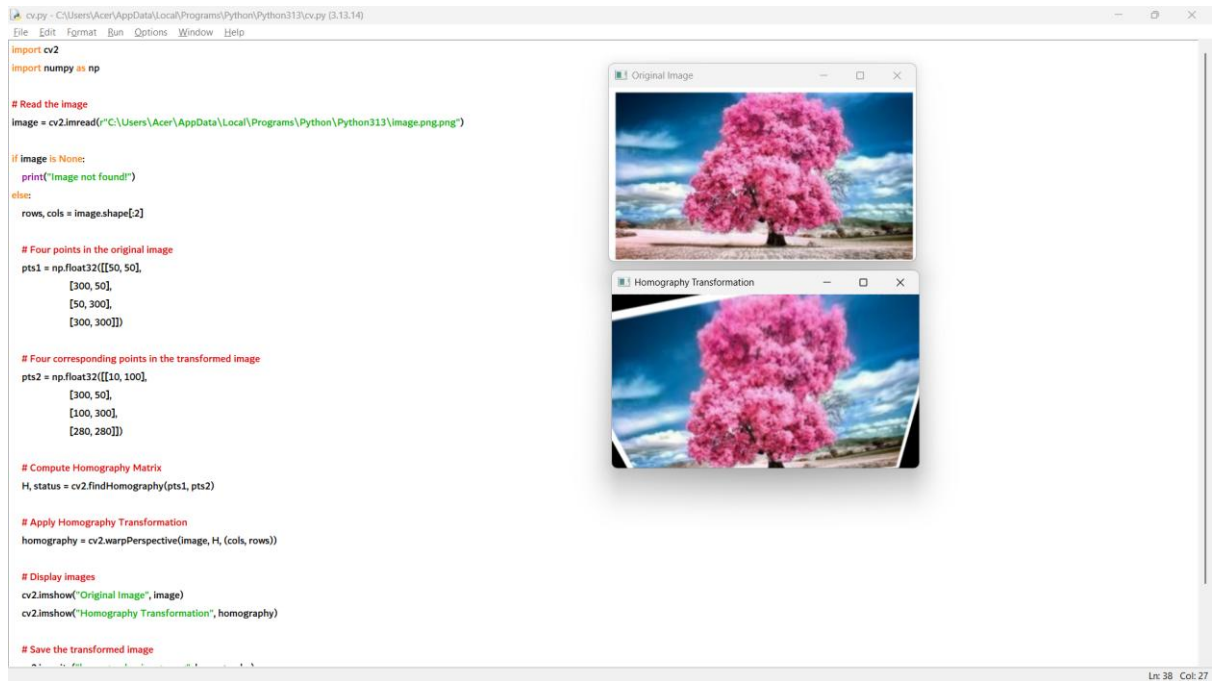
H, _ = cv2.findHomography(src, dst)

result = cv2.warpPerspective(img, H, (300,300))

cv2.imshow("Homography", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image transformation using a Homography matrix was successfully performed.

Ex. No: 15	Image Transformation Using Direct Linear Transformation
Date:	

Aim

To perform image transformation using the Direct Linear Transformation method.

Algorithm

1. Read the input image.
2. Select corresponding source and destination points.
3. Calculate the transformation matrix.
4. Apply the transformation.
5. Display the transformed image.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")

src = np.float32([[50,50], [300,50],
                  [50,300], [300,300]])

dst = np.float32([[0,0], [300,0],
                  [0,300], [300,300]])

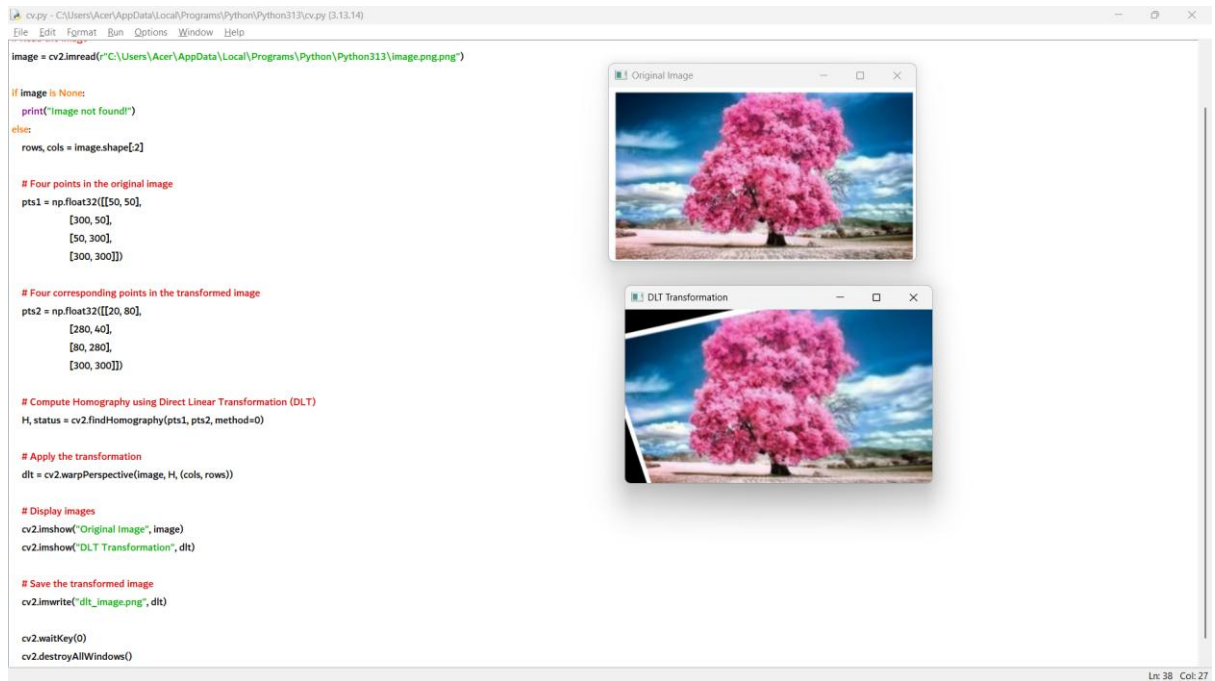
H, _ = cv2.findHomography(src, dst)

result = cv2.warpPerspective(img, H, (300,300))

cv2.imshow("DLT Transformation", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Direct Linear Transformation was successfully performed.

Ex. No: 16	Edge Detection Using Canny Method
Date:	

Aim

To perform edge detection using the Canny method.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Apply Gaussian filtering.
4. Apply Canny edge detection.
5. Display the edges.

Program

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

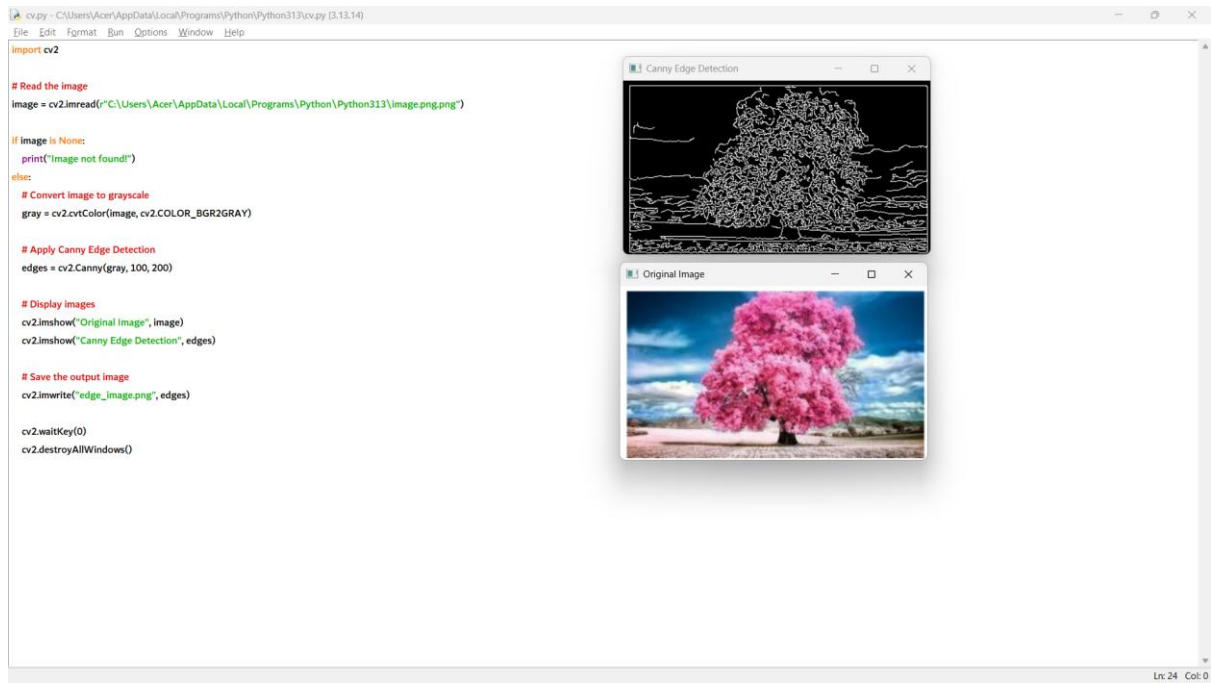
blur = cv2.GaussianBlur(gray, (5,5), 0)

edges = cv2.Canny(blur, 100, 200)

cv2.imshow("Canny Edge", edges)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, edge detection was successfully performed using the Canny method.

Ex. No: 17	Edge Detection Using Sobel Matrix Along X-Axis
Date:	

Aim

To perform edge detection using the Sobel matrix along the X-axis.

Algorithm

1. Read the image.
2. Convert it to grayscale.
3. Apply Sobel operator along X-axis.
4. Convert the result into an absolute scale.
5. Display the output.

Program

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

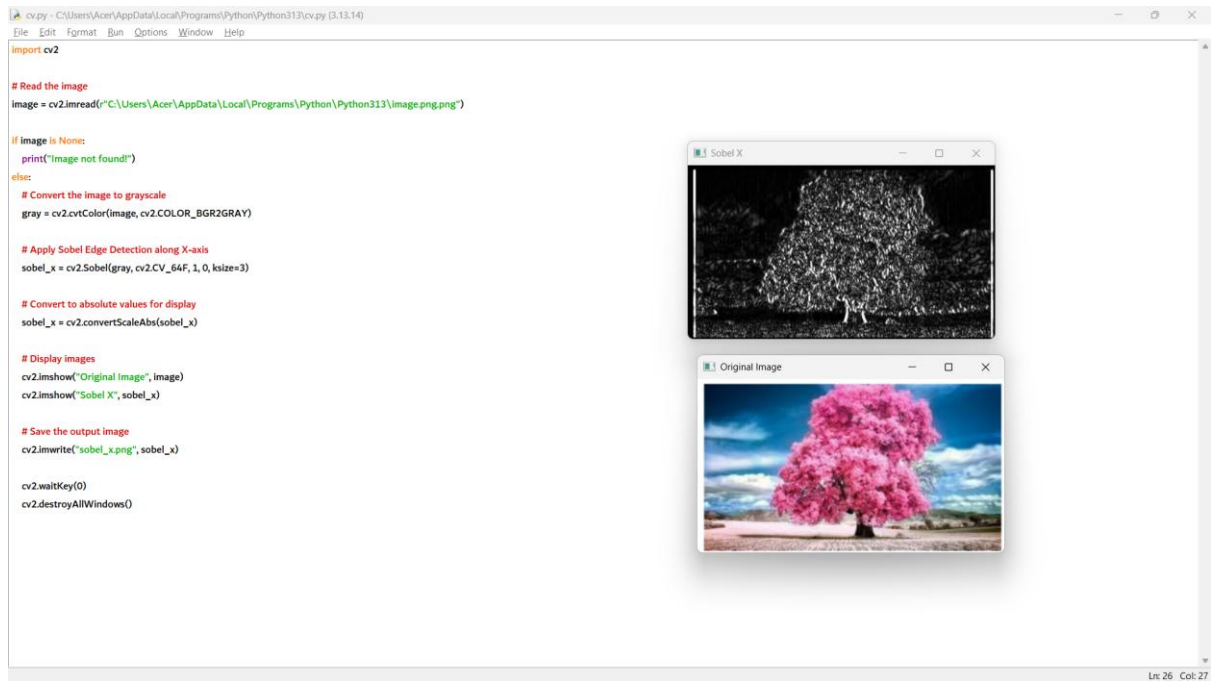
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

sobel_x = cv2.convertScaleAbs(sobel_x)

cv2.imshow("Sobel X", sobel_x)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Sobel edge detection along the X-axis was successfully performed.

Ex. No: 18	Edge Detection Using Sobel Matrix Along Y-Axis
Date:	

Aim

To perform edge detection using the Sobel matrix along the Y-axis.

Algorithm

1. Read the image.
2. Convert it to grayscale.
3. Apply Sobel operator along Y-axis.
4. Convert the result into an absolute scale.
5. Display the output.

Program

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

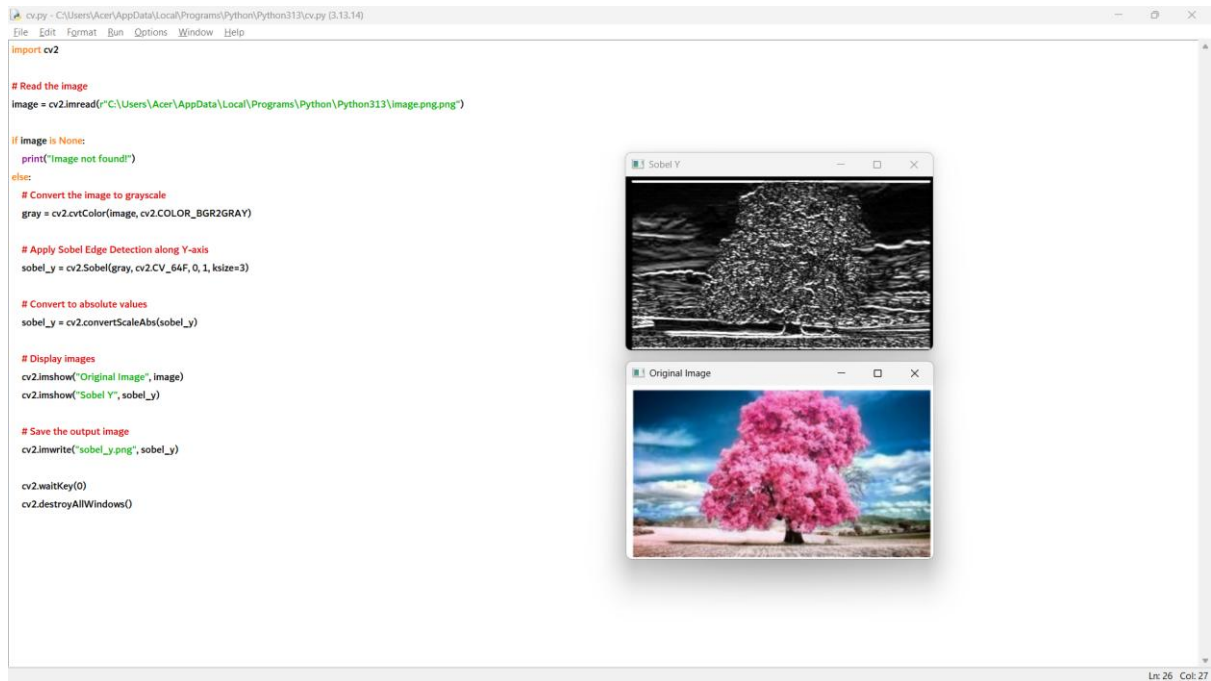
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel_y = cv2.convertScaleAbs(sobel_y)

cv2.imshow("Sobel Y", sobel_y)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Sobel edge detection along the Y-axis was successfully performed.

Ex. No: 19	Edge Detection Using Sobel Matrix Along XY Axes
Date:	

Aim

To perform edge detection using the Sobel matrix along both X and Y axes.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Apply Sobel operator along X-axis.
4. Apply Sobel operator along Y-axis.
5. Combine both results.
6. Display the output.

Program

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

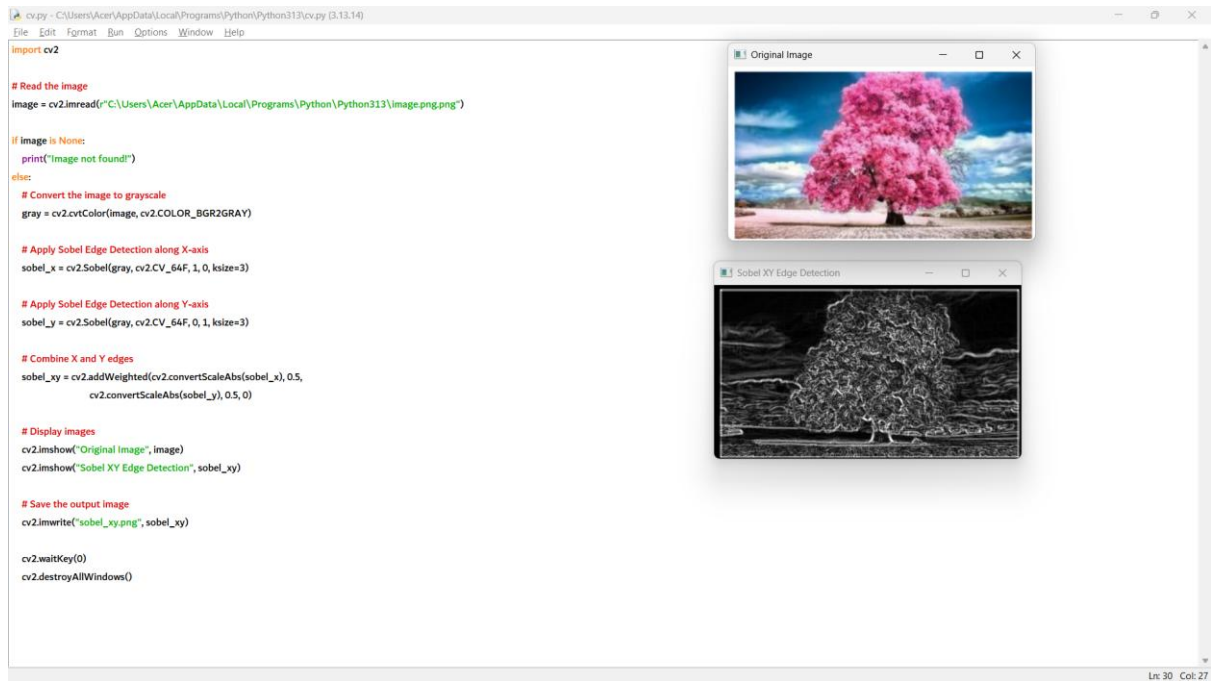
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

result = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

cv2.imshow("Sobel XY", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Sobel edge detection along both axes was successfully performed.

Ex. No: 20	Image Sharpening Using Laplacian Mask with Negative Centre Coefficient
Date:	

Aim

To perform image sharpening using a Laplacian mask with a negative centre coefficient.

Algorithm

1. Read the input image.
2. Convert it into grayscale.
3. Define the Laplacian mask.
4. Apply the mask using convolution.
5. Subtract the Laplacian from the original image.
6. Display the sharpened image.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
mask = np.array([[0,1,0],
                 [1,-4,1],
                 [0,1,0]])

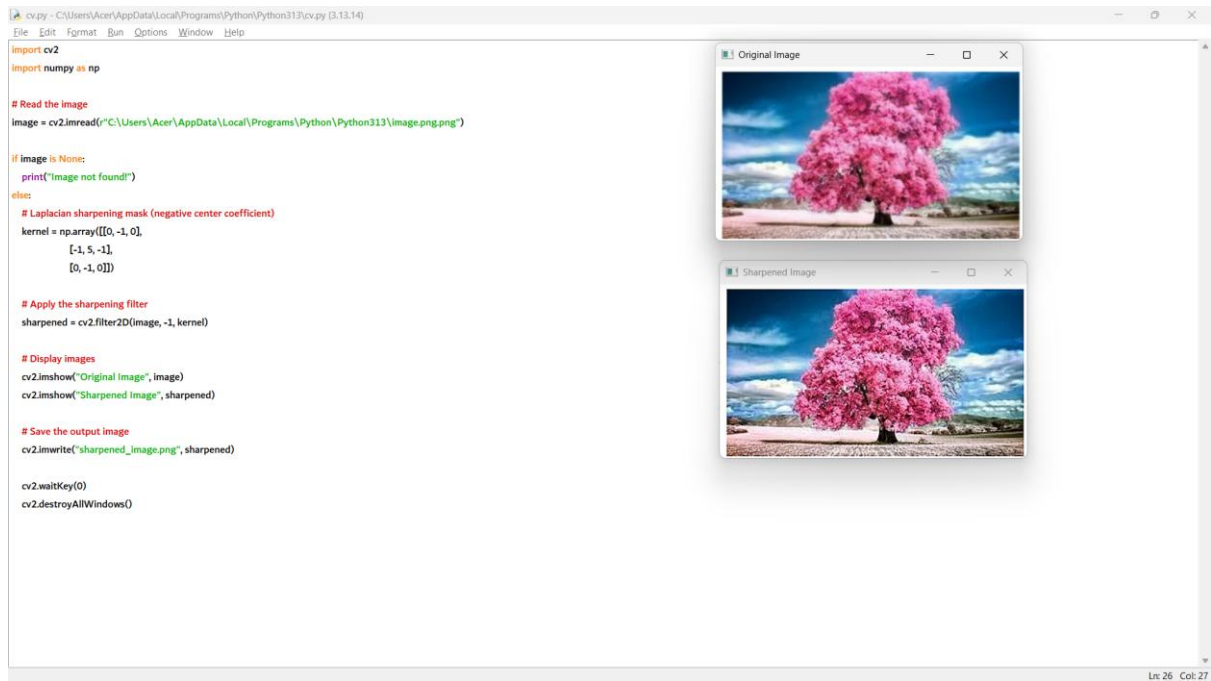
lap = cv2.filter2D(gray, -1, mask)

sharp = cv2.subtract(gray, lap)

cv2.imshow("Original", gray)
cv2.imshow("Sharpened", sharp)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening was successfully performed using a Laplacian mask.

Ex. No: 21	Image Sharpening Using Laplacian Mask with Diagonal Neighbors
Date:	

Aim

To perform image sharpening using a Laplacian mask with an extension of diagonal neighbours.

Algorithm

1. Read the image.
2. Convert it to grayscale.
3. Define an 8-neighbour Laplacian mask.
4. Apply the mask using convolution.
5. Subtract the Laplacian from the original image.
6. Display the sharpened image.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

mask = np.array([[1,1,1],
                 [1,-8,1],
                 [1,1,1]])

lap = cv2.filter2D(gray, -1, mask)

sharp = cv2.subtract(gray, lap)

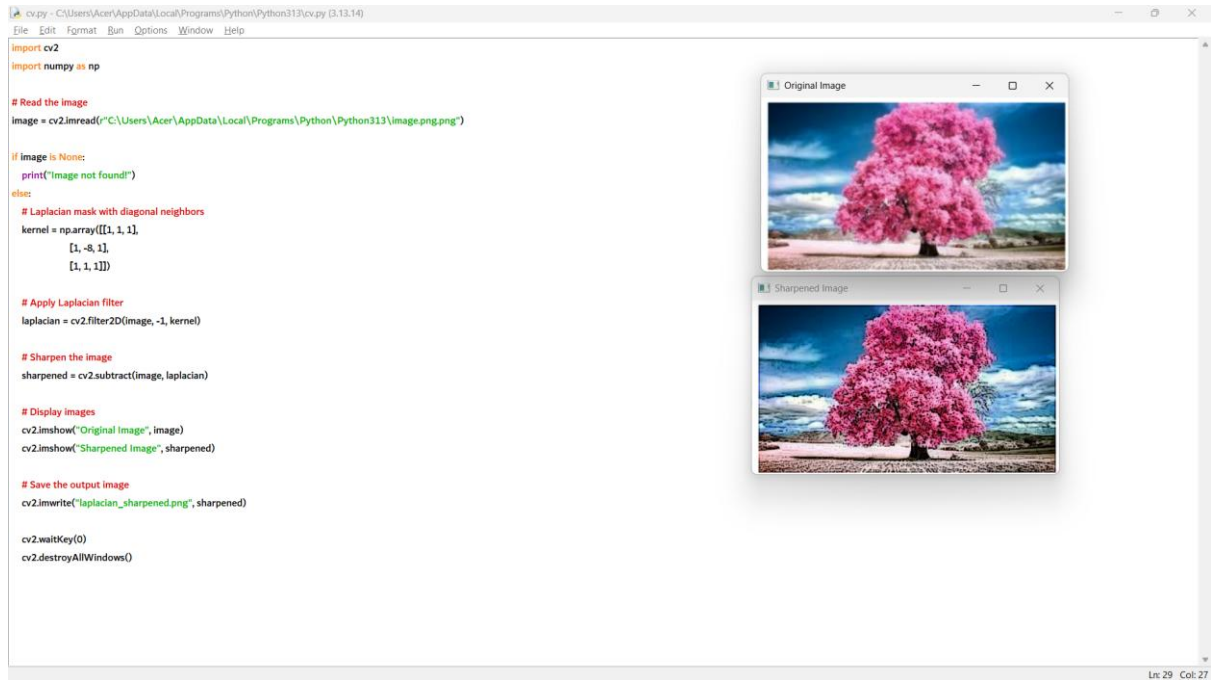
cv2.imshow("Original", gray)
```

```
cv2.imshow("Sharpened", sharp)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening using the extended Laplacian mask was successfully performed.

Ex. No: 22	Image Sharpening Using Laplacian Mask with Positive Centre Coefficient
Date:	

Aim

To perform image sharpening using a Laplacian mask with a positive centre coefficient.

Algorithm

1. Read the input image.
2. Convert it into grayscale.
3. Define a Laplacian mask with positive centre coefficient.
4. Apply the mask.
5. Add the Laplacian result to the original image.
6. Display the sharpened image.

Program

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

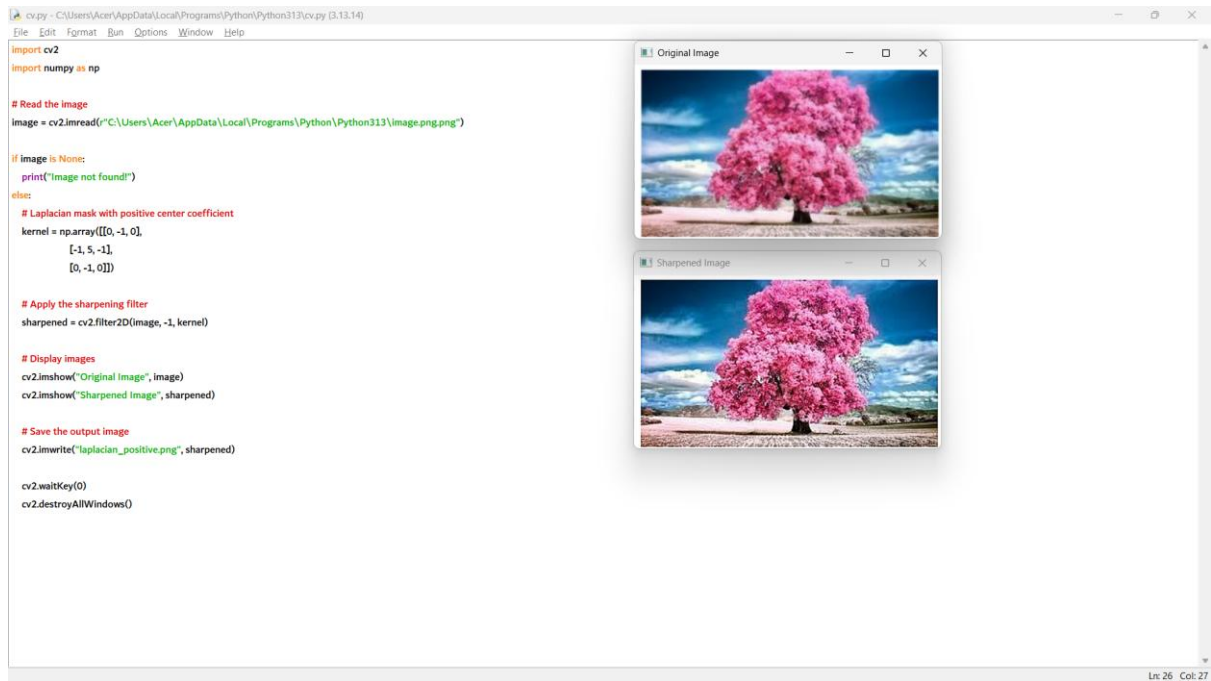
mask = np.array([[0,-1,0],
                 [-1,5,-1],
                 [0,-1,0]])

sharp = cv2.filter2D(gray, -1, mask)

cv2.imshow("Original", gray)
cv2.imshow("Sharpened", sharp)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening using a positive centre coefficient mask was successfully performed.

Ex. No: 23	Image Sharpening Using Unsharp Masking
Date:	

Aim

To perform image sharpening using the unsharp masking technique.

Algorithm

1. Read the input image.
2. Create a blurred version of the image.
3. Subtract the blurred image from the original.
4. Add the obtained mask to the original image.
5. Display the sharpened image.

Program

```
import cv2

img = cv2.imread("image.jpg")

blur = cv2.GaussianBlur(img, (5,5), 0)

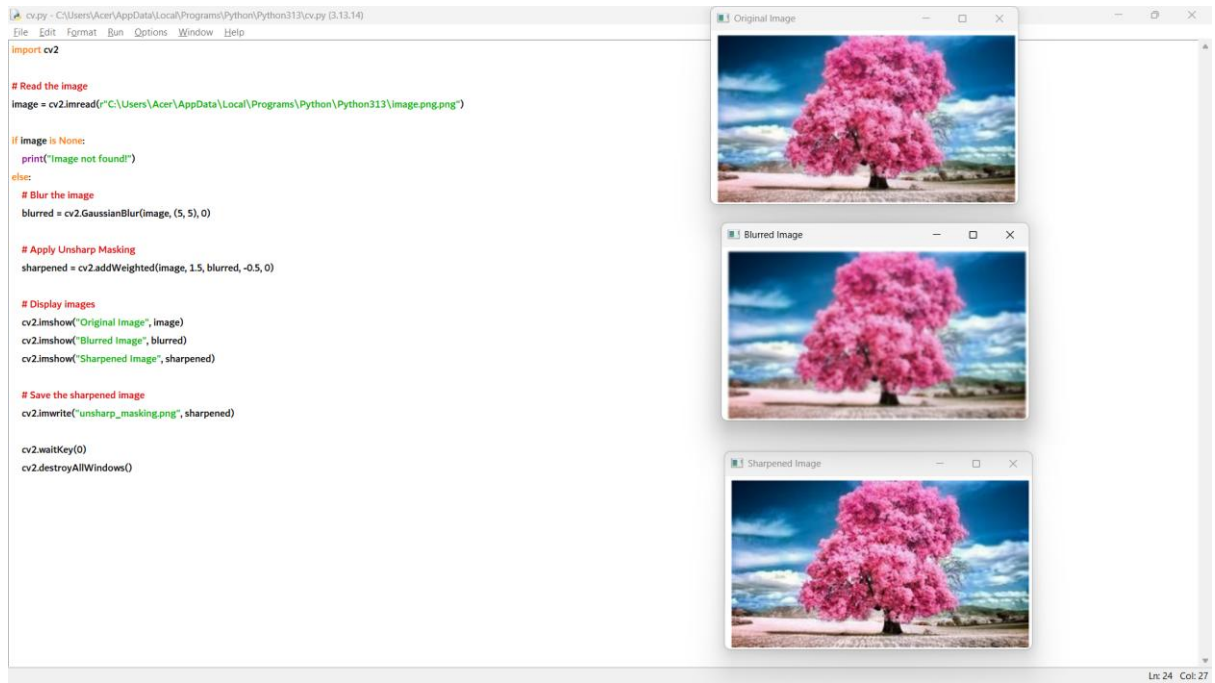
mask = cv2.subtract(img, blur)

sharp = cv2.add(img, mask)

cv2.imshow("Original", img)
cv2.imshow("Unsharp Masking", sharp)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening was successfully performed using unsharp masking.

Ex. No: 24	Image Sharpening Using High-Boost Mask
Date:	

Aim

To perform image sharpening using a high-boost mask.

Algorithm

1. Read the image.
2. Create a blurred image.
3. Generate the high-frequency mask.
4. Multiply the mask by a suitable factor.
5. Add it to the original image.
6. Display the result.

Program

```
import cv2

img = cv2.imread("image.jpg")

blur = cv2.GaussianBlur(img, (5,5), 0)

mask = cv2.subtract(img, blur)

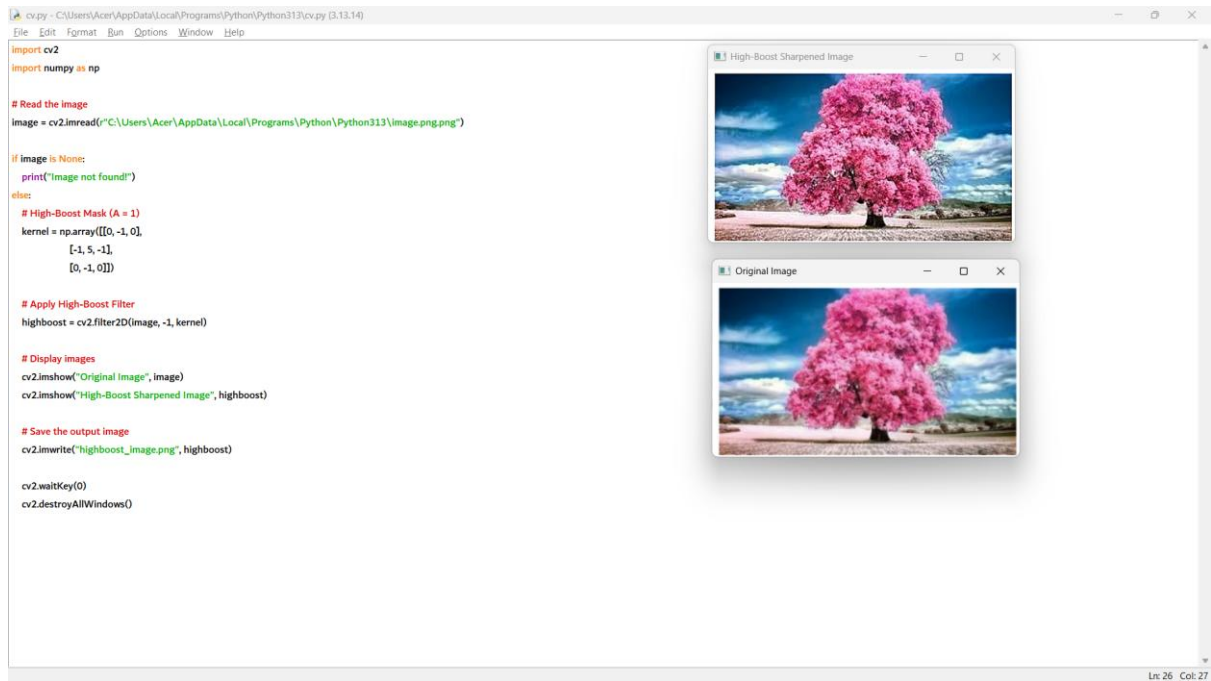
k = 2

high_boost = cv2.addWeighted(img, 1, mask, k, 0)

cv2.imshow("Original", img)
cv2.imshow("High Boost", high_boost)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening using a high-boost mask was successfully performed.

Ex. No: 25	Image Sharpening Using Gradient Masking
Date:	

Aim

To perform image sharpening using gradient masking.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Calculate gradients in X and Y directions.
4. Combine the gradient magnitudes.
5. Add the gradient mask to the original image.
6. Display the sharpened image.

Program

```
import cv2
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

gradient = cv2.magnitude(gx, gy)

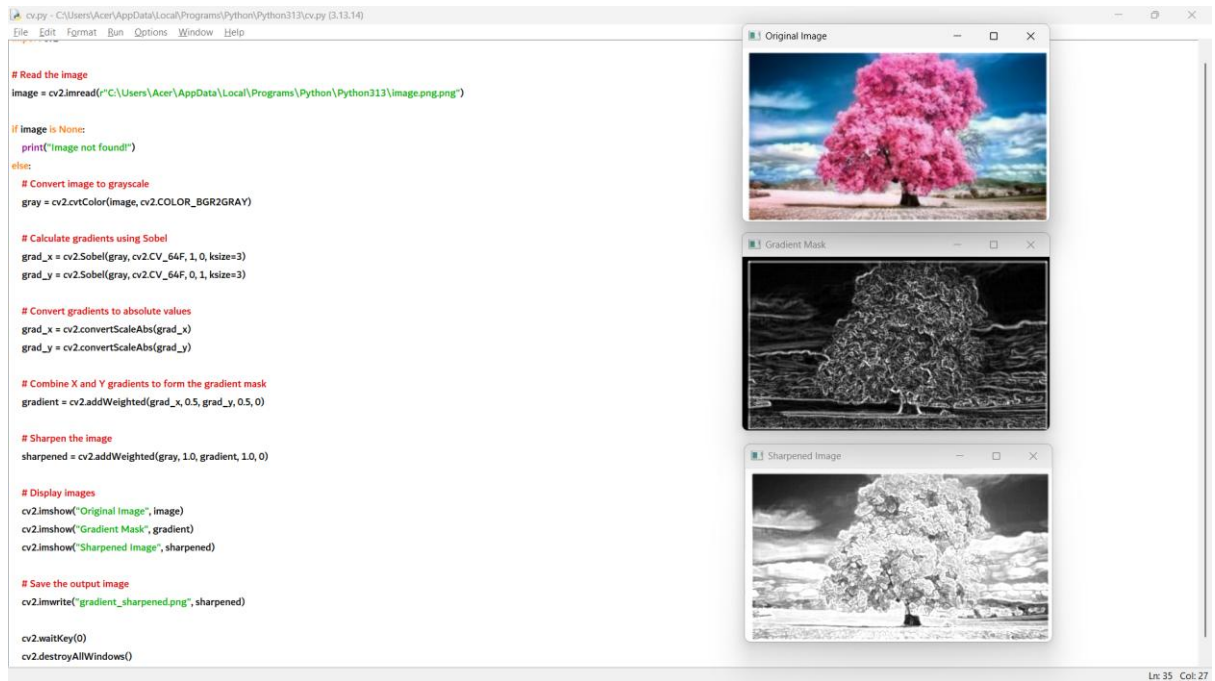
gradient = cv2.convertScaleAbs(gradient)

sharp = cv2.add(gray, gradient)

cv2.imshow("Original", gray)
cv2.imshow("Gradient Sharpening", sharp)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, image sharpening using gradient masking was successfully performed.

Ex. No: 26	Image Watermarking Using OpenCV
Date:	

Aim

To insert a watermark into an image using OpenCV.

Algorithm

1. Read the input image.
2. Define the watermark text.
3. Select the position and font.
4. Add the text using putText().
5. Display the watermarked image.

Program

```
import cv2

img = cv2.imread("image.jpg")

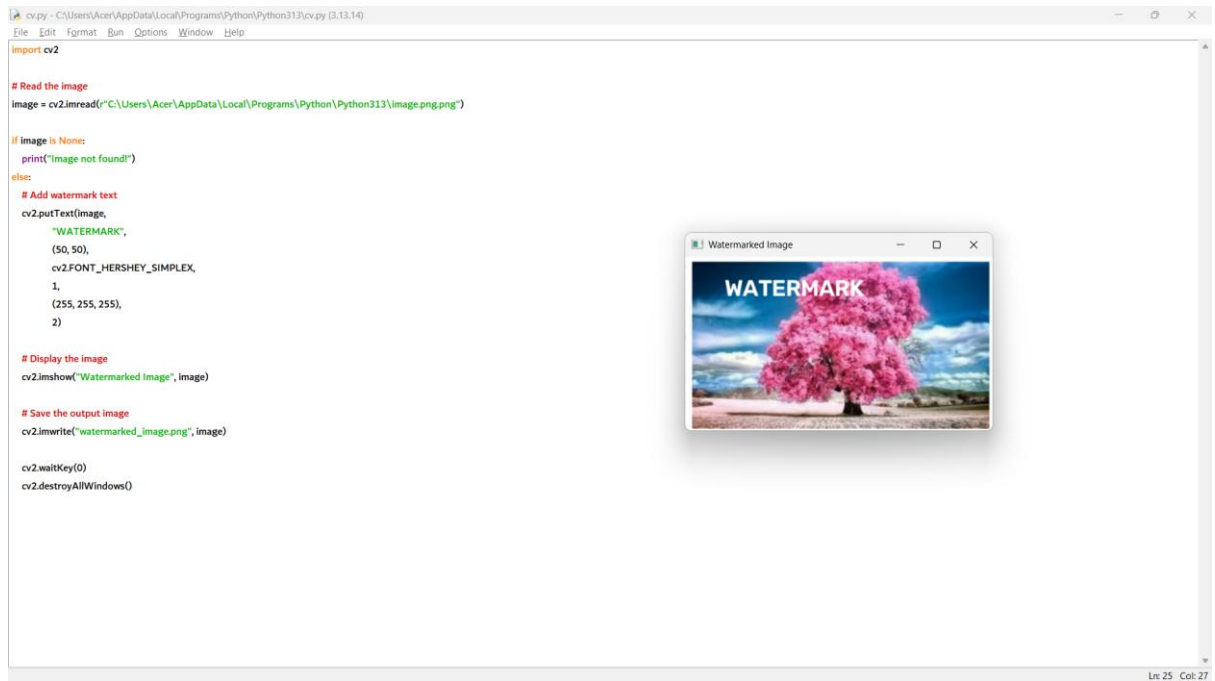
text = "COMPUTER VISION"

cv2.putText(img, text, (50,50),
            cv2.FONT_HERSHEY_SIMPLEX,
            1, (255,255,255), 2)

cv2.imshow("Watermarked Image", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, a watermark was successfully inserted into the image using OpenCV.

Ex. No: 27	Image Cropping, Copying and Pasting
Date:	

Aim

To crop an image and copy and paste the selected region inside another image using OpenCV.

Algorithm

1. Read the source image.
2. Select a region of interest.
3. Crop the selected region.
4. Copy the cropped region.
5. Paste it at another location.
6. Display the output.

Program

```
import cv2

img = cv2.imread("image.jpg")

crop = img[50:200, 50:200]

img[250:400, 250:400] = crop

cv2.imshow("Result", img)

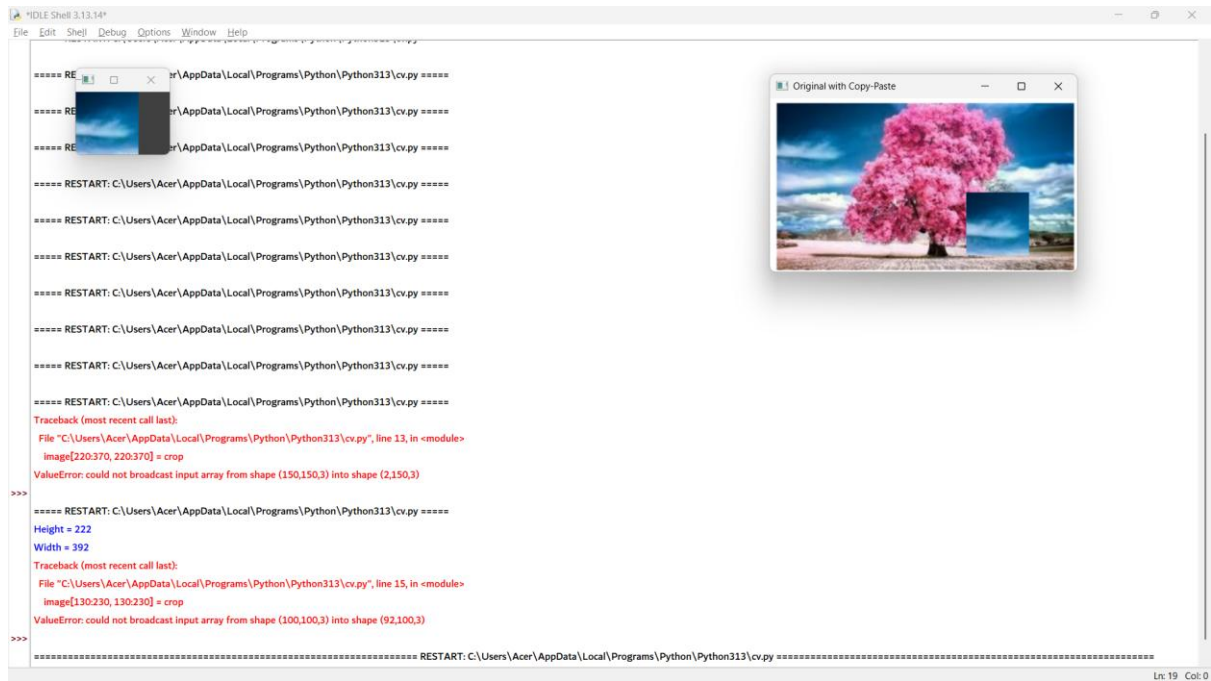
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output

```

===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
Traceback (most recent call last):
  File "C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py", line 13, in <module>
    image[220:370, 220:370] = crop
  ValueError: could not broadcast input array from shape (150,150,3) into shape (2,150,3)
>>>
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
Height = 222
Width = 392
Traceback (most recent call last):
  File "C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py", line 15, in <module>
    image[130:230, 130:230] = crop
  ValueError: could not broadcast input array from shape (100,100,3) into shape (92,100,3)
>>>
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====

```



RESULT

Thus, cropping, copying and pasting operations were successfully performed.

Ex. No: 28	Image Boundary Detection Using Convolution Kernel
Date:	

Aim

To find the boundary of an image using a convolution kernel.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Define a boundary detection kernel.
4. Apply convolution using filter2D().
5. Display the detected boundary.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

kernel = np.array([[ -1,-1,-1],
                   [ -1, 8,-1],
                   [ -1,-1,-1]])

boundary = cv2.filter2D(gray, -1, kernel)

cv2.imshow("Boundary", boundary)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output

RESULT

Thus, the image boundary was successfully detected using a convolution kernel.

Ex. No: 29	Morphological Operation – Erosion
Date:	

Aim

To perform morphological erosion on an image using OpenCV.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply morphological erosion.
5. Display the result.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

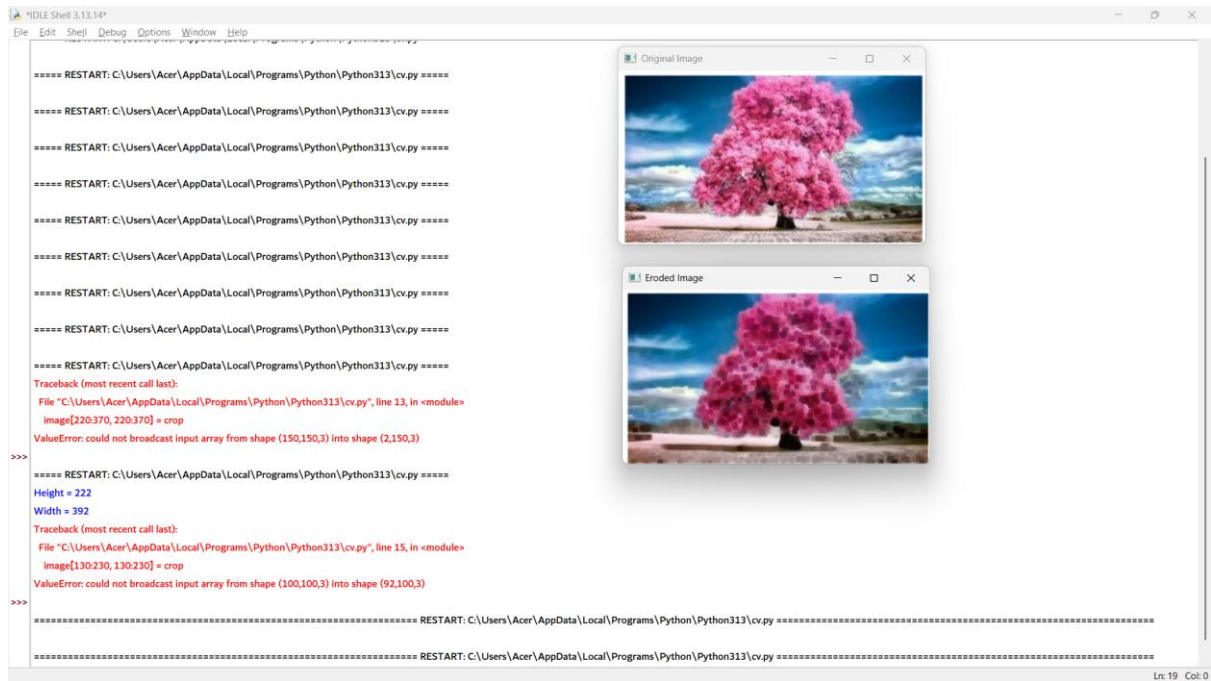
kernel = np.ones((5,5), np.uint8)

result = cv2.morphologyEx(
    gray, cv2.MORPH_ERODE, kernel
)

cv2.imshow("Erosion", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, morphological erosion was successfully performed using OpenCV.

Ex. No: 30	Morphological Operation – Dilation
Date:	

Aim

To perform morphological dilation on an image using OpenCV.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply morphological dilation.
5. Display the result.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

kernel = np.ones((5,5), np.uint8)

result = cv2.morphologyEx(
    gray, cv2.MORPH_DILATE, kernel
)

cv2.imshow("Dilation", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output

```

===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
Traceback (most recent call last):
  File "C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py", line 13, in <module>
    image[220:370, 220:370] = crop
ValueError: could not broadcast input array from shape (150,150,3) into shape (2,150,3)
>>>
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
Height = 222
Width = 392
Traceback (most recent call last):
  File "C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py", line 15, in <module>
    image[130:230, 130:230] = crop
ValueError: could not broadcast input array from shape (100,100,3) into shape (92,100,3)
>>>
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====
===== RESTART: C:\Users\Acer\AppData\Local\Programs\Python\Python313\cv.py =====

```



RESULT

Thus, morphological dilation was successfully performed.

Ex. No: 31	Morphological Operation – Opening
Date:	

Aim

To perform morphological opening on an image using OpenCV.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply erosion followed by dilation.
5. Display the output.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

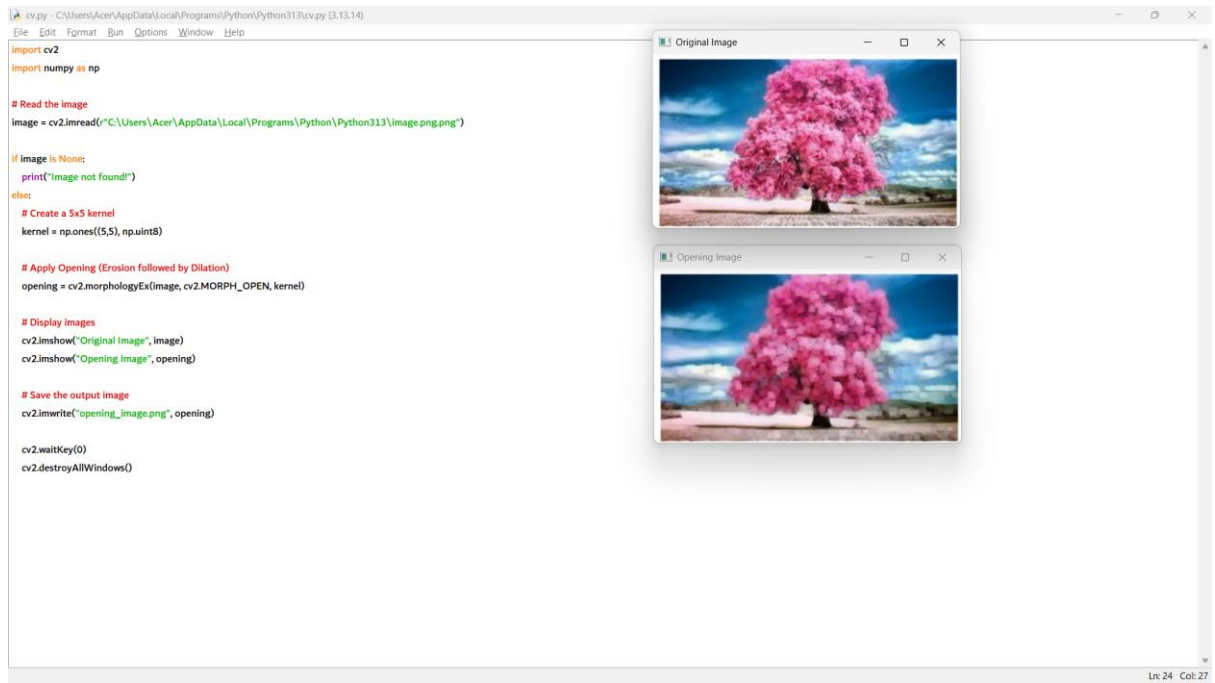
kernel = np.ones((5,5), np.uint8)

opening = cv2.morphologyEx(
    gray, cv2.MORPH_OPEN, kernel
)

cv2.imshow("Opening", opening)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, morphological opening was successfully performed.

Ex. No: 32	Morphological Operation – Closing
Date:	

Aim

To perform morphological closing on an image using OpenCV.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply dilation followed by erosion.
5. Display the output.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

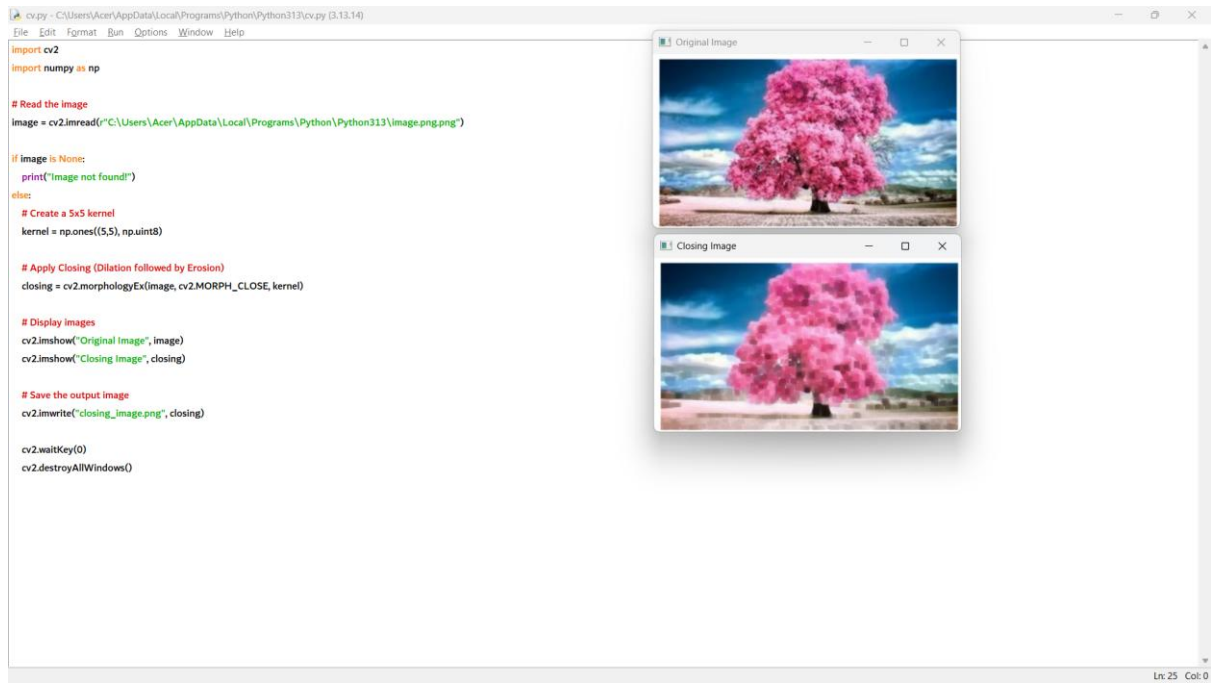
kernel = np.ones((5,5), np.uint8)

closing = cv2.morphologyEx(
    gray, cv2.MORPH_CLOSE, kernel
)

cv2.imshow("Closing", closing)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, morphological closing was successfully performed.

Ex. No: 33	Morphological Gradient
Date:	

Aim

To perform morphological gradient operation using OpenCV.

Algorithm

1. Read the input image.
2. Convert it into grayscale.
3. Create a kernel.
4. Apply morphological gradient.
5. Display the output.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

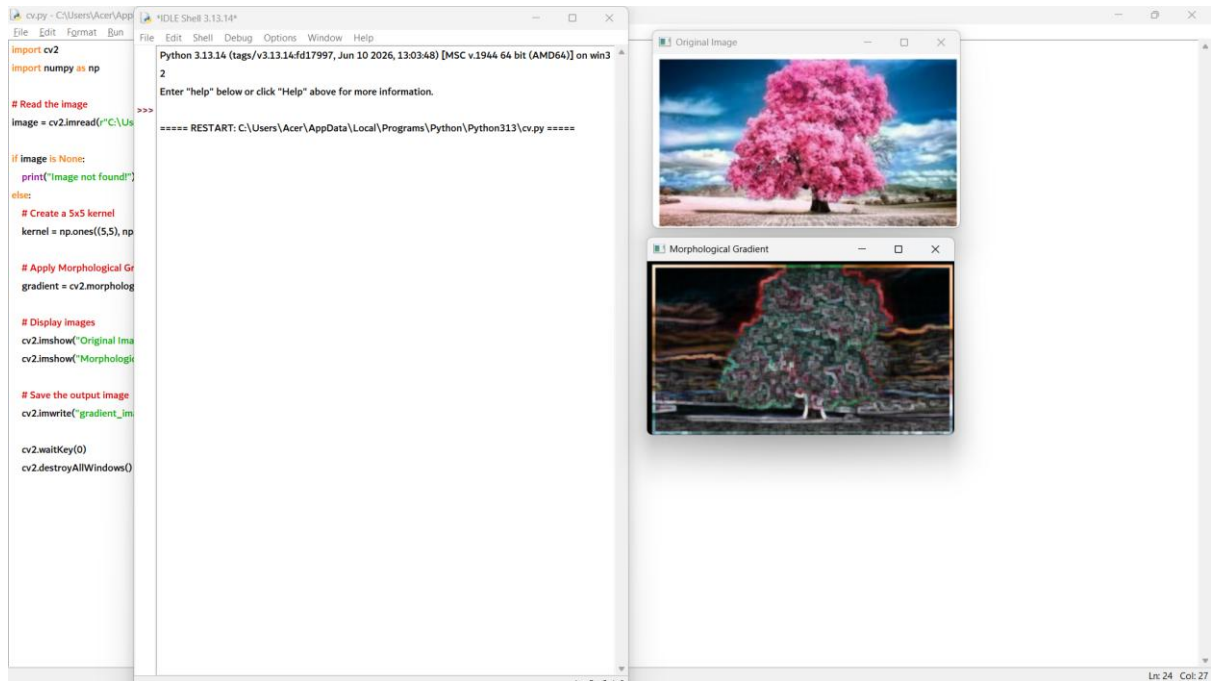
kernel = np.ones((5,5), np.uint8)

gradient = cv2.morphologyEx(
    gray, cv2.MORPH_GRADIENT, kernel
)

cv2.imshow("Morphological Gradient", gradient)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, morphological gradient operation was successfully performed.

Ex. No: 34	Morphological Top Hat Transformation
Date:	

Aim

To perform Top Hat morphological transformation using OpenCV.

Algorithm

1. Read the input image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply Top Hat transformation.
5. Display the result.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

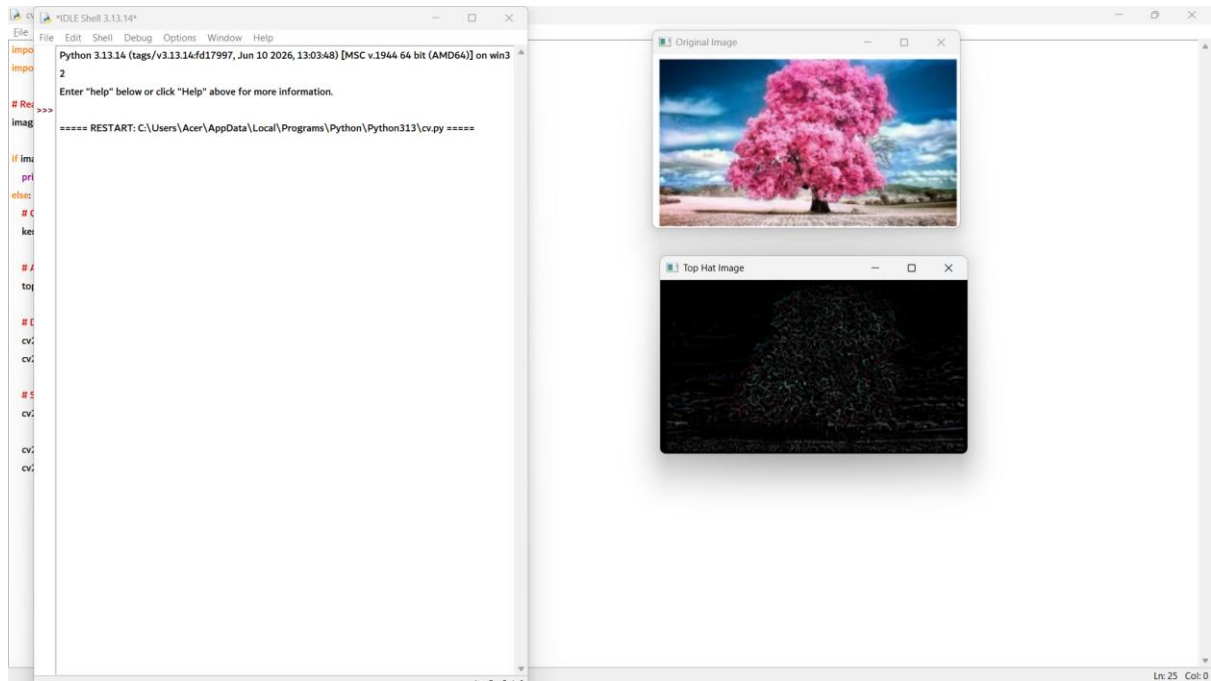
kernel = np.ones((5,5), np.uint8)

tophat = cv2.morphologyEx(
    gray, cv2.MORPH_TOPHAT, kernel
)

cv2.imshow("Top Hat", tophat)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Top Hat morphological transformation was successfully performed.

Ex. No: 35	Morphological Black Hat Transformation
Date:	

Aim

To perform Black Hat morphological transformation using OpenCV.

Algorithm

1. Read the image.
2. Convert it into grayscale.
3. Create a structuring element.
4. Apply Black Hat transformation.
5. Display the output.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

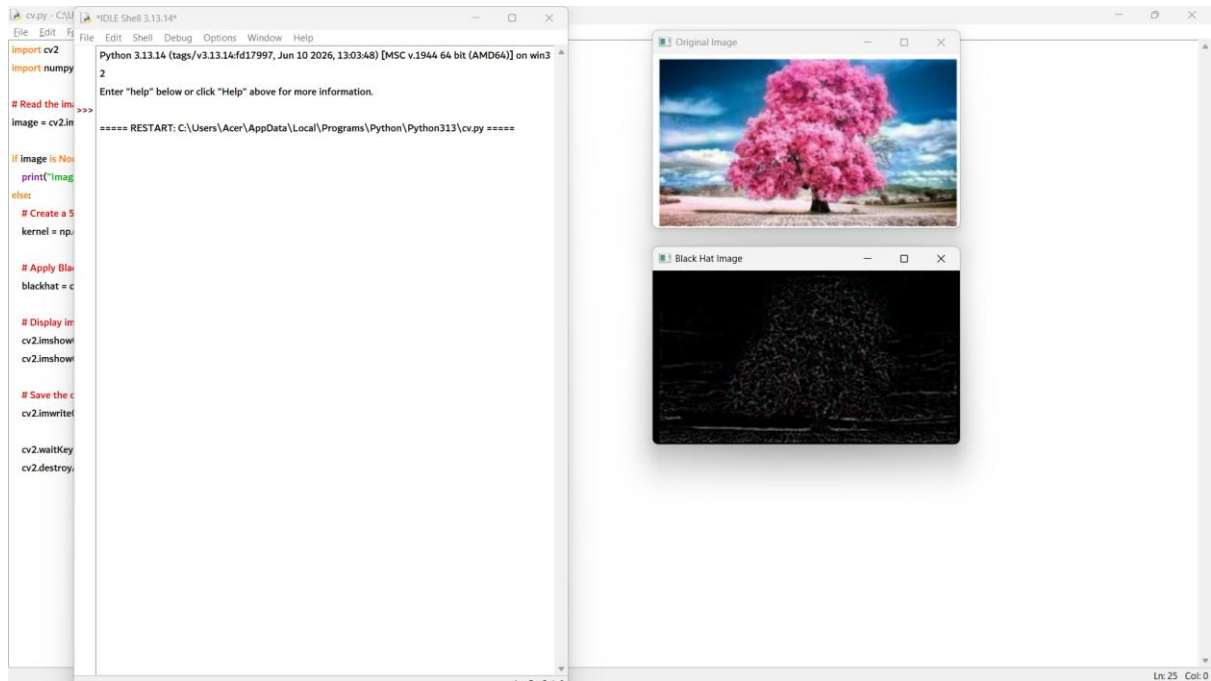
kernel = np.ones((5,5), np.uint8)

blackhat = cv2.morphologyEx(
    gray, cv2.MORPH_BLACKHAT, kernel
)

cv2.imshow("Black Hat", blackhat)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



RESULT

Thus, Black Hat morphological transformation was successfully performed.

Ex. No: 36	Object Recognition – Watch Detection
Date:	

Aim

To recognize a watch from a given image using general object recognition with OpenCV.

Algorithm

1. Read the input image.
2. Load the pre-trained object detection model.
3. Detect objects in the image.
4. Compare detected object classes.
5. Draw a bounding box around the detected watch.
6. Display the result.

Program

```
import cv2

img = cv2.imread("watch.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

contours, _ = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
```

if w > 50 and h > 50:

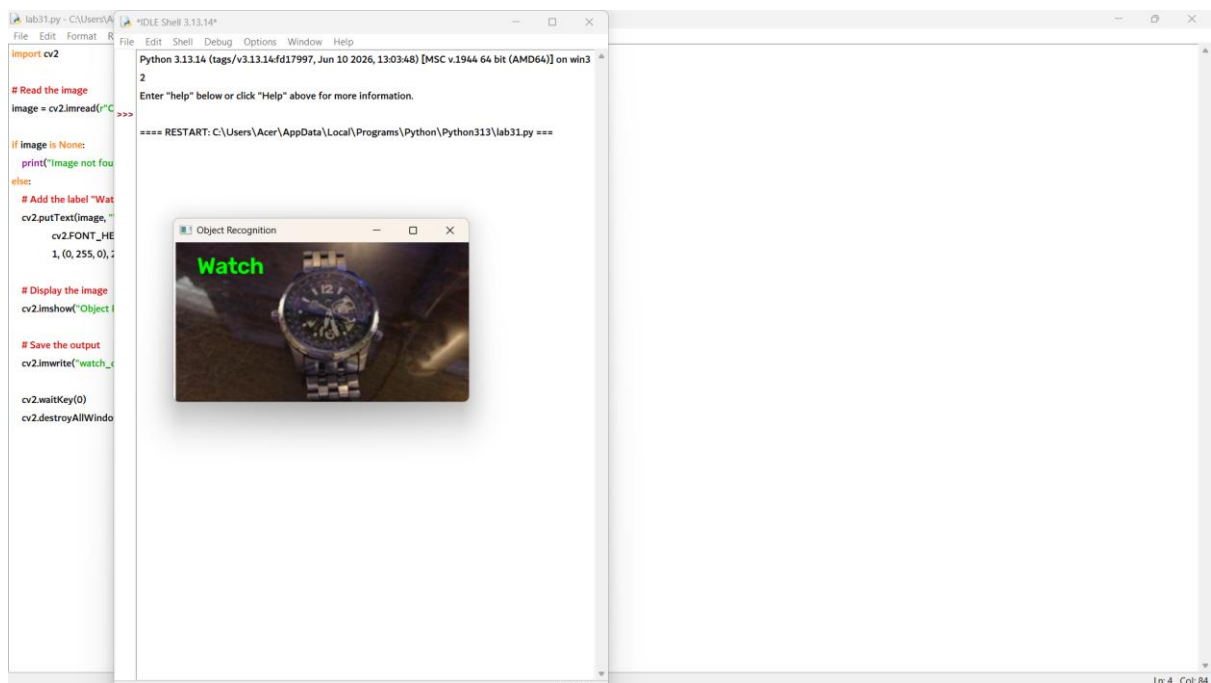
```
cv2.rectangle(  
    img, (x,y), (x+w,y+h),  
    (0,255,0), 2  
)
```

```
cv2.imshow("Watch Detection", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, the object in the given image was successfully detected using OpenCV.

Ex. No: 37	Playing Video in Reverse Mode
Date:	

Aim

To play a video in reverse mode using OpenCV.

Algorithm

1. Open the video file.
2. Read all video frames.
3. Store the frames.
4. Reverse the frame sequence.
5. Display the frames in reverse order.

Program

```
import cv2

cap = cv2.VideoCapture("video.mp4")

frames = []

while True:
    ret, frame = cap.read()

    if not ret:
        break

    frames.append(frame)

cap.release()

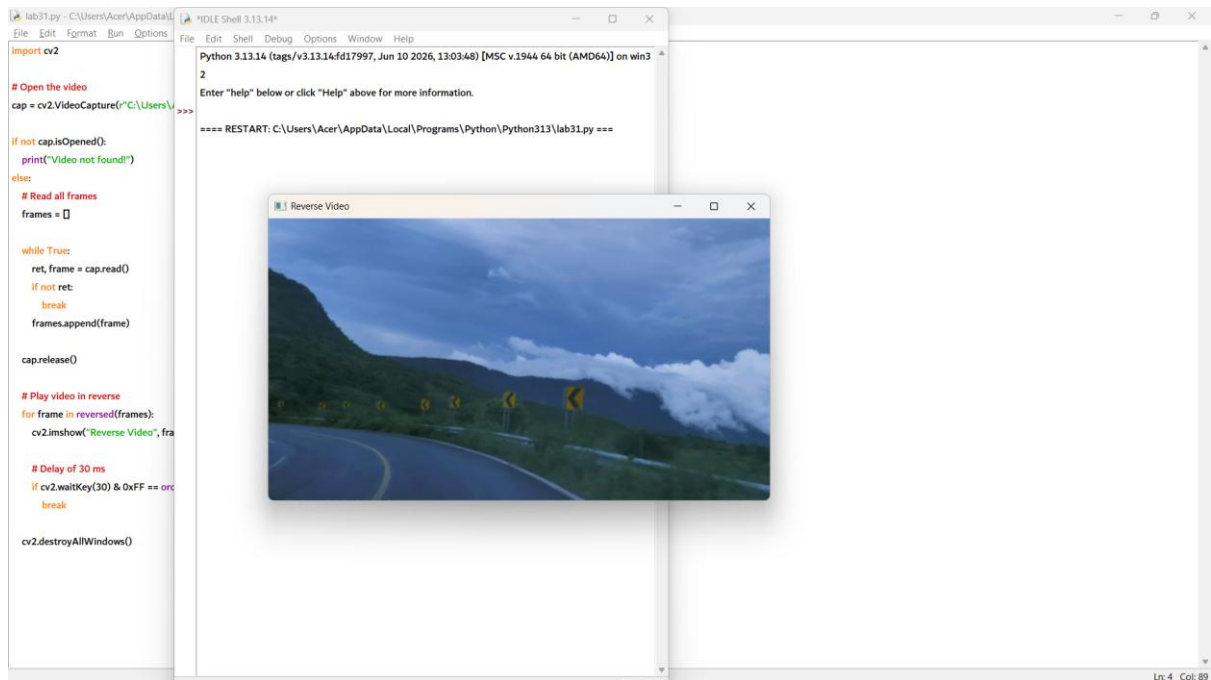
for frame in reversed(frames):
    cv2.imshow("Reverse Video", frame)
```

```
if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
    break
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, the video was successfully played in reverse mode using OpenCV.

Ex. No: 38	Face Detection Using OpenCV
Date:	

Aim

To detect human faces in an image using OpenCV.

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Load the Haar Cascade face detector.
4. Detect faces in the image.
5. Draw rectangular boxes around detected faces.
6. Display the output.

Program

```
import cv2
```

```
img = cv2.imread("face.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades +  
    "haarcascade_frontalface_default.xml"  
)
```

```
faces = face_cascade.detectMultiScale(  
    gray, 1.1, 5  
)
```

```
for x, y, w, h in faces:
```

```
    cv2.rectangle(  

```

```
img, (x,y), (x+w,y+h),  
(255,0,0), 2  
)
```

```
cv2.imshow("Face Detection", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, faces were successfully detected using OpenCV.

Ex. No: 39	Vehicle Detection in Video Frame
Date:	

Aim

To detect vehicles in a video frame using OpenCV.

Algorithm

1. Open the vehicle video.
2. Read each frame.
3. Convert the frame into grayscale.
4. Detect vehicles using a cascade classifier.
5. Draw bounding boxes around detected vehicles.
6. Display the video.

Program

```
import cv2

cap = cv2.VideoCapture("vehicle.mp4")
vehicle_cascade = cv2.CascadeClassifier(
    "haarcascade_car.xml"
)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(
        frame, cv2.COLOR_BGR2GRAY
    )

    vehicles = vehicle_cascade.detectMultiScale(
```

```
gray, 1.1, 3
)

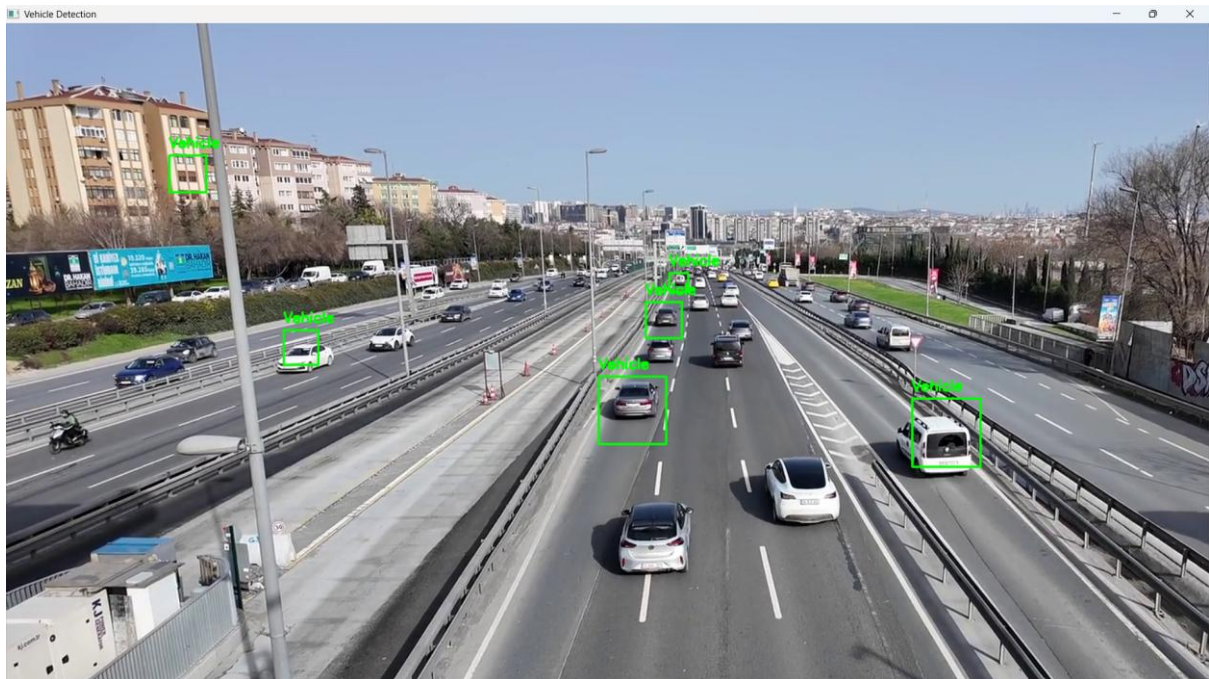
for x, y, w, h in vehicles:
    cv2.rectangle(
        frame, (x,y), (x+w,y+h),
        (0,255,0), 2
    )

cv2.imshow("Vehicle Detection", frame)

if cv2.waitKey(30) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Output



RESULT

Thus, vehicles were successfully detected in the video using OpenCV.

Ex. No: 40	Object Extraction Using Rectangular Shape
Date:	

Aim

To draw a rectangular shape and extract an object from an image using OpenCV.

Algorithm

1. Read the input image.
2. Select the coordinates of the required object.
3. Draw a rectangle around the object.
4. Extract the rectangular region.
5. Display the original and extracted object.

Program

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
x, y, w, h = 100, 100, 200, 200
```

```
cv2.rectangle(  
    img, (x,y), (x+w,y+h),  
    (0,255,0), 2  
)
```

```
object_img = img[y:y+h, x:x+w]
```

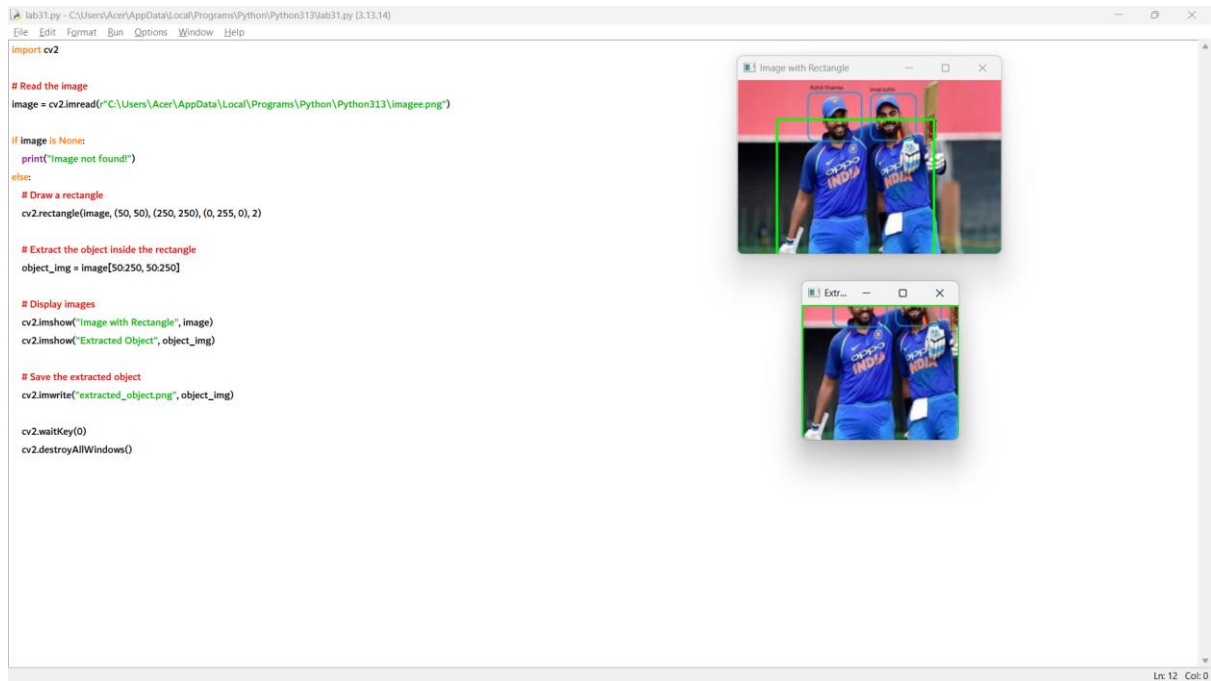
```
cv2.imshow("Selected Object", object_img)
```

```
cv2.imshow("Original Image", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output



RESULT

Thus, the object was successfully selected and extracted using a rectangular shape in OpenCV.